Improving Computational Efficiency of Wind Energy Resource Assessment through Novel Statistical Wake and Access Models



Kaiya Raby
RegNo: 202268958

A thesis presented in fulfilment of the requirements for the degree of Masters by Research

Supervisors: Dr Abbas Mehrad Kazemi Amiri, Dr Edward Hart and Dr Julian Feuchtwang

School of Electronic and Electrical Engineering,
The University of Strathclyde,
Royal College Building,
204 George Street,
Glasgow,
G1 1RX

July 2024

Acknowledgments

I would like to acknowledge the funding bodies that enabled me to be part of the Wind and Marine Energy Systems and Structures (WAMESS) CDT, including ESPRC, Siemens-Gamesa and the John Anderson Research Award. I would also like to thank my supervisory team and the wider department for their help, support and guidance throughout my studies and research. In particular, I would like to acknowledge my primary supervisor, Abbas Mehrad Kazemi Amiri, who gave me much patience, assistance and valuable insight. Additionally, I would like to express my gratitude to Andrew Smith, Ciara McAvoy, and Edward Hart, who went to great lengths to enable the continuation of my studies through difficult circumstances and without whose invaluable support this work would not have been possible.

Contents

1 Access Modelling **1**
- 1.1 Motivation . . . 1
- 1.2 Background . . . 1
- 1.3 Methodology . . . 2
  - 1.3.1 Continuous-Time Models . . . 3
  - 1.3.2 Discrete-Time Models . . . 4
  - 1.3.3 Markov Chain Model . . . 5
  - 1.3.4 DTMC Model . . . 5
- 1.4 Results . . . 7
  - 1.4.1 Geospatial Analysis . . . 7
  - 1.4.2 Goodness-of-fit Tests . . . 8
  - 1.4.3 Percentage Error . . . 8
  - 1.4.4 Confidence Intervals . . . 9
- 1.5 Use in O&M Cost Prediction . . . 10
- 1.6 Conclusions . . . 11

2 Wake Modelling **12**
- 2.1 Introduction . . . 1
- 2.2 Prior Models . . . 1
  - 2.2.1 The Jensen Model . . . 1
  - 2.2.2 The Frandsen Model . . . 2
- 2.3 Current Model . . . 4
  - 2.3.1 The Bastankhah Porte-Agel (BPA) Model . . . 4
  - 2.3.2 Reformulation . . . 5
  - 2.3.3 Current Implementation in StrathFarm . . . 7
  - 2.3.4 Reformulation . . . 9
- 2.4 Implementation and Testing . . . 10
- 2.5 Results . . . 11
- 2.6 Limitations . . . 12
- 2.7 Conclusions . . . 12

Bibliography **13**

Appendices **16**
- A: Access Modelling . . . 17
- B: Wake Modelling . . . 35

Abstract

With the rapid growth of the wind energy sector and, consequently, site development, accurate and efficient resource assessment is becoming increasingly relevant. In-depth assessment requires a multitude of complex considerations, including the study of highly stochastic environmental conditions such as wind field and wave patterns. Understanding these conditions is essential to many Key Performance Indicators such as Energy Yield, O&M cost, accessibility and Levelized Cost of Energy (LCOE). To achieve this, computationally complex techniques such as Monte Carlo simulations are typically employed, however these methods are rarely feasible for comparing long timespans or large areas. This thesis focuses on improving the computational efficiency of resource assessment for wind energy by producing novel methods in two key areas of environmental forecasting: access prediction and wake modelling. The improved efficiency enablesquicker resource assessment and comparison of large areas to optimise site selection.

The first section proposes a highly accurate and efficient Markov Chain model, based on a weather and accessibility state space, to estimate several key accessibility traits, including the probability of instant access and estimated delay time of repairs due to weather. This model has competitive accuracy with existing probabilistic methods but with improved computational cost, allowing for assessing a three-year North Sea dataset in an order of minutes. This model may be used instead of more intensive Monte-Carlo simulations whilst still producing high accuracy and associated confidence.

The second section derives and solves a Fourier approximation of a Gaussian wake based on the Bastankhah Porte-Agel model [1], to be implemented in Strathfarm, a holistic wind farm simulation software aiming to produce medium-fidelity results in real-time. The current implementation requires computationally intensive look-up tables based on polynomial approximations. The proposed method derives an efficiently calculable polynomial in terms of farm conditions only, removing the need for extremely large look-up table storage and access. This new method significantly improves efficiency whilst maintaining the accuracy of the current methodology within 10%.

Chapter 1

Access Modelling

Abstract

This chapter aims to outline and computationally implement stochastic methods to estimate accessibility traits of potential offshore assets. The significance of accessibility in estimating *Levelised Cost of Energy* is outlined in Section 1.1. Section 1.2 then discusses the existing methods for estimating key accessibility indicators, most commonly those involving Monte Carlo simulations [2, 3, 4, 5, 6, 7, 8], and highlights their flaws largely relating to computational complexity, and poor scaling due to non-linear order.

Alternative methods are then derived and described in Section 1.3. Firstly, an empirical method is established to calculate observed instant access probability and delay times due to weather. This method can provide a relatively efficient linear order exploratory analysis to identify accessibility features and validate the additional methods. This method is demonstrated for a 10-year 0.08-degree specific dataset of the North Sea. Secondly, the distribution-based methods developed by Feuchtwang et al (2013) are implemented and tuned in Python using improved maximum likelihood estimation methods. Finally, a Discrete-Time Markov Chain (DTMC) is developed, inclusive of associated confidence intervals able to attach certainty to forecasted estimates using adjusted variance for future intra-year variation.

These methods are then validated and compared using three datasets in Section 1.4. The results demonstrate that each method is of linear order and aims to allow for large-scale multi-year data analysis. All methods are competitive with existing models in terms of predictive accuracy, with percentage errors below 10% even for datasets limited to 2 years, compared to the suggested minimum length of 10 years. For larger datasets, it is shown that the DTMC model can produce analytical confidence intervals for forecasted values, containing the observed hindcast values over a range of scenarios. Section 1.5 then discusses the implications of these results for calculations of Operation and Maintenance (O&M) cost, and overall conclusions are given in Section 1.6.

1.1 Motivation

With wind energy providing an increasing share of energy production, reducing costs is vital to ensure the affordability of the energy supply. Inaccessibility may reduce availability and energy production by up to 35% in some locations [4], as well as increase Operation and Maintenance (O&M) costs due to long wait times and more expensive operations. These delays may, in turn, increase the failure rate, further increasing both downtime and repair costs [9]. The following section aims to emphasize that these effects pose a significant risk to the value of a site and, therefore, highlight the importance of understanding and quantifying their impact.

One of the most widely accepted and utilised methods for value assessment of an asset or location is the *levelised cost of energy* (LCOE) defined as

$$LCOE = \frac{\frac{CAPEX}{(1+i)^T} + \sum_{t=1}^{T} \frac{OPEX_t}{(1+i)^{T-t}}}{\sum_{t=1}^{T} \frac{E_t}{(1+i)^{T-t}}},$$

where

$$\begin{aligned} CAPEX &= \text{ The capital expenditure of a site in GBP} \\ OPEX_t &= \text{ the operational expenditure of a site in GBP over year } t \\ T &= \text{ the total number of years a site operates for} \\ i &= \text{ the interest rate} \\ E_t &= \text{ the energy produced in year t.} \end{aligned}$$

Thus, this metric represents the cost of an asset or site per unit of energy produced over its lifetime. The accessibility of an asset affects two key parts of this equation:

- OPEX - an inaccessible asset will incur higher operations and maintenance costs for several reasons. When an asset fails, if it is more inaccessible, we either have to use a vessel more equipped to handle the harsher conditions, which may be more expensive, or the repairs must be on hold until the weather conditions are safe. This waiting period may lead to knock-on damage and failures.
- E_t - the yielded energy may be decreased due to longer downtime waiting for safe conditions to repair.

Hence, the ability to quantify accessibility is highly relevant to those conducting site or resource assessments.

1.2 Background

Whilst highly significant to availability estimates, limited literature exists specifically modelling accessibility and related outputs (e.g. expected weather delay time). The majority of the available literature incorporates computationally complex Monte Carlo (MC) simulations [3, 2, 10, 7, 4, 11]. Although providing high-fidelity results, simulations of this nature are intensive and require significant time and computational resources, especially where access is one of many inputs to a broader model for cost or availability. One such model is proposed in Papadopoulos et al (2022), suggesting leveraging of Numerical Weather Predictions (NWP)s or Machine Learning (ML) methods to inform a Holistic Opportunity-Based Strategy (HOST) for the prediction of weather windows for opportunistic maintenance [9]. NWPs are too complex to run in real-time and may delay large simulations such as this, especially for geospatial analyses that compare many points simultaneously. Similarly, ML models such as Long Short-Term Memory Neural Networks show high efficacy, comparable with MC models for significant wave height prediction [12]. Still, they are likely to exhibit slower performance than probabilistic models, which require far less intensive training.

To address this, Feuchtwang (2009) proposed a purely probabilistic and highly efficient model to calculate accessibility traits such as expected weather delay time and probability of instant access [13]. This model requires only one training step, where several Weibull distributions are fitted to the data efficiently,

allowing all further estimates to be calculated almost instantaneously without requiring additional training. Further model explanation and expansion can be found in Feuchtwang and Infielf (2013) [14]. In a later study, these methods are combined with a holistic O&M model to calculate associated costs [15], demonstrating that these estimates may be incorporated easily into broader systems. This varies from other probabilistic and Weibull-based models for access modelling, such as the multi-step probabilistic wave height forecasting model used by Zhang et al (2021) [2], which are either used in combination with a more complex system such as an MC simulation or do not produce point estimates and outputs suitable for further investigation in the same way (e.g. time-series regression for future prediction). This report aims to computationally implement these discussed methods in Python and compare them to a proposed Discrete-Time Markov Chain Model. The goal is to meet the same criteria: reducing computational cost compared to Monte Carlo (MC) models while still accounting for the complex stochastic nature of weather parameters, such as significant wave height.

Whilst there have been several examples of Markov-Chain Models within availability modelling, these generally consist of a similar structure: a Continuous-Time Markov Chain with a set parameter for expected wait time due to weather delay or accessibility [4, 10, 7, 8, 11]. A key example of this can be seen in Centeno-Telleria et al (2022), where an extensive O%M model is developed using a Continuous-Time Markov Chain availability model, with accessibility modelled as the average wait time until a weather window. This is based on previous work by Dinwoodie (2012) and Dinwoodie and McMillan (2014) who used the same architecture with accessibility traits modelled by Autoregressive Time Series forecasting. Similarly, Huang et al (2017) build a broad CTMC with expected weather delay treated as a Poisson random variable. These methods vary from the proposed model in two ways. Firstly, these settings treat the time domain as continuous. It is often the case that our threshold for safe access or travel is based on significant wave height, or on time-series data that is either averaged or taken in set intervals (e.g. every 10 minutes). In each of these cases, the time domain analyzed is discrete, and the model used should reflect this. Thus, this report works in the discrete-time domain. Secondly, these models include accessibility as a parameter of the CTMC as a component of the expected repair wait time. Conversely, the proposed model uses a Markov Chain to model the accessibility traits themselves.

This report aims to outline a set of methods for estimating two key values associated with offshore accessibility: probability of instant access (i.e. no delay), and expected time until the next access window. These methods include continuous-time distribution-based models developed by Feuchtwang and Infield (2013) [14], a Discrete-Time Markov Chain, and an Empirical Model. Section 1.3 fully derives and describes each method in depth. The efficacy of point and confidence interval estimates and the computation time are then assessed and compared in Section 1.4 using three reference data sets.

1.3 Methodology

Following the occurrence of a fault, maintenance cannot be conducted until the weather is considered safe. We define the safety of an operation in terms of the time required, t_{req}, and the threshold wave weight, H_{th}, which the significant wave height must fall below for the vessel to conduct the operation safely. The event tree followed to meet these safety requirements is shown in Figure 1.1.

The above event tree implies that at a given time,, following an event, we are in exactly one of three distinct situations listed below.

- Delay Type A: Weather conditions unsuitable for access
- Delay Type B: Weather conditions suitable for access, but insufficient time remaining to carry out repair
- No Delay: Access possible, and sufficient time remaining to carry out repair

Each of the models outlined aim to calculate the probability of being in the third state (i.e. experiencing no delay) at a given time and the expected wait time until this point is reached (i.e. expected delay). Both of these outputs are considered to be a function of the two described operation characteristics, H_{th} and t_{req}.

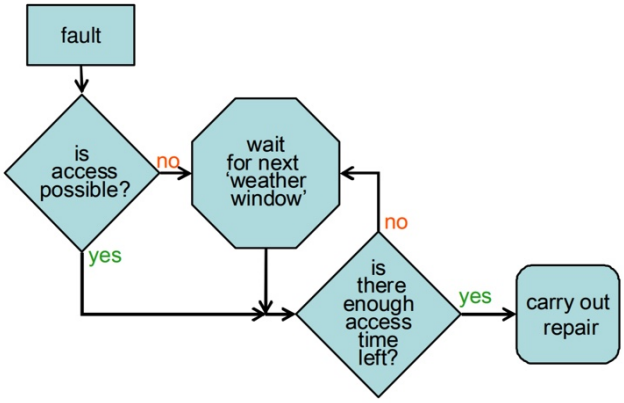


Figure 1.1: Simplified Event Tree for Offshore Repairs [16]

1.3.1 Continuous-Time Models

The continuous-time models, defined by Feuchtwang (2009) [13], calculate accessibility using three Weibull models fitted to the observed distributions of significant wave height, calm duration, and storm duration. For some threshold wave height, H_{th}, the storm duration times are given by time spent with significant wave height exceeding the threshold height. Conversely, calm durations are continuous times between exceedances of the threshold height. A three-parameter distribution is used for significant wave height, whilst two-parameter models are used to model the calm and storm durations.

Several distribution fitting methods were trialled and assessed via likelihood and AIC and BIC scores. The best-fitting methods were the Method of Moments defined by Cran (1988) [17], and Maximum Likelihood Estimation (MLE) approaches, with MLE performing slightly more accurately.

The cumulative distribution function and the fitted distributions for all three distributions using the MLE method may be seen in Figure 1.2. These estimated fits were well fitting visually, as well as through Kolmogorov-Smirnoff and Chi-square goodness-of-fit tests with p-values for each of the six tests falling below the threshold of a 0.01 significance level.

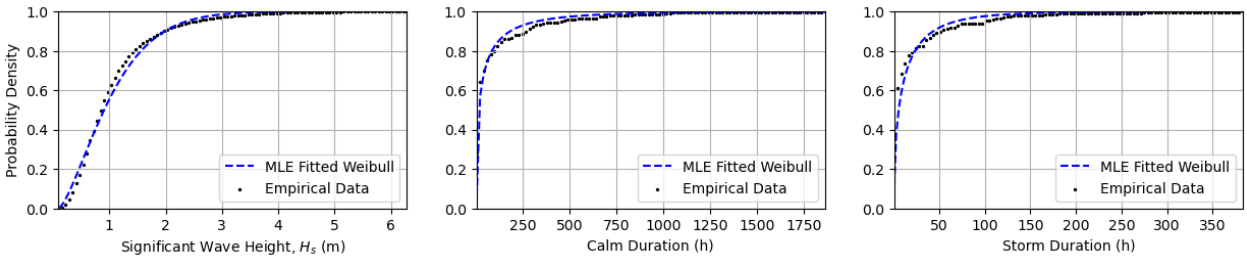


Figure 1.2: Weibull Distribution Fits

These distributions were then used to calculate the access probability as [14]

$$P_0 = \left[1 - P(H_{th})\right] \cdot \left[1 - M_{qn}(H_{th}, t_{req}) - \frac{Q_n(H_{th}, t_{req}) t_{req}}{\tau_n(H_{th})}\right], \tag{1.1}$$

where $P(H)$ is the probability of wave height exceeding H, $\tau_n(H)$ is the mean calm duration of wave heights falling below H, $\tau_x(H)$ is the mean storm duration of wave heights exceeding H, and $Q_n(H, t)$ is the probability of the calm duration exceeding t.

The expected delay time is given by [14]

$$
\begin{aligned}
E =& P(H_{th}) \cdot M_{qqx}(H_{th}) \cdot \tau_x(H_{th}) && (1.2)\\
&+ \frac{(1-P(H_{th})^2}{P(H_{th})} \cdot M_{qqn}(H_{th}) \cdot \tau_x(H_{th}) && (1.3)\\
&+ P(H_{th}) \cdot Q_n(H_{th}, t_{req}) \cdot t_{req} && (1.4)\\
&+ \big(1-P(H_{th})\big) \cdot M_{qn}(H_{th}, t_{req} \cdot \tau_x(H_{th}) && (1.5)\\
&+ \Big[P(H_{th}) + \big(1-P(H_{th})\big)\, M_{qn}(H_{th}, t_{req}\Big] \cdot t_{req} && (1.6)\\
&+ \frac{\Big[P(H_{th}) + \big(1-P(H_{th})\big)\, M_{qn}(H_{th}, t_{req})\Big]^2 \cdot \tau_x(H_{th})}{P(H_{th}) \cdot Q_n(H_{th}, t_{req})}, && (1.7)
\end{aligned}
$$

where $M_{qn}(H,t)$ is the normalized partial first moment, from 0 to t, of the calm duration probability density function, and M_{qqx} and M_{qqn} are half the normalised complete second moment of the storm and calm duration probability density functions respectively.

1.3.2 Discrete-Time Models

The typical data set comprises time-series wave-height data, $\mathbf{H} = \{H_0, H_1, \cdots, H_T\}$, indexed by time. Let us define two Bernoulli state space variables, X_i and Y_i. Y_i refers to the weather conditions within this time period, either 'calm' if the significant wave height is below the threshold limit, or 'stormy' if this limit is exceeded. Specifically,

$$
Y_i = \begin{cases} 0 & \text{if } H_i < H_{th} \text{ (calm)} \\ 1 & \text{if } H_i \geq H_{th} \text{ (stormy).} \end{cases} \tag{1.8}
$$

It follows that X_i defined by

$$
X_i = \begin{cases} 0 & \text{if } \sum_{j=0}^{t_{req}-1} Y_{i+j} = 0 \\ 1 & \text{if } \sum_{j=0}^{t_{req}-1} Y_{i+j} > 0. \end{cases} \tag{1.9}
$$

denotes the accessibility, where an asset is considered accessible at time i, (i.e. $X_i = 0$) if the conditions are calm for the next t_{req} periods.

Empirical Model

The empirical model calculates the probability of immediate access simply as μ_X, the mean of the accessibility states. This is equivalent to the proportion of time intervals with instant access. The expected time until access is given by the average of the delay vector, $\mathbf{D}$, the time until the next accessible state. These are calculated as outlined in Algorithm 1.

Algorithm 1: Empirical Method

```
input: H, H_th, t_req
output: X

for i = 0, 1, ··· , t_req − 1
    if H_i ≥ H_th
        Y_i=1
    else
        Y_i=0
end

if Σ_{j=0}^{t_req−1} Y_{i+j} = 0
    X_i=1
end

counter = 0
for i = 0, 1, ··· , N
```

```
17        if X_i=1
18            counter = 0
19        else
20            counter = counter+1
21            break
22        D_i = counter
23    end
24
25    P_0 = average(X)
26    λ_exp = average(D)
```

The order of this algorithm is linear, as can be determined within the pseudocode, resulting in a fast and efficient process. The confidence interval of expected delay may then be calculated using the mean, μ_D, and standard deviation, σ_D of the delay vector as

$$\lambda_L = \mu_D - \frac{z_{\frac{\alpha}{2}}\sigma_D}{\sqrt{N}} \tag{1.10}$$

$$\lambda_U = \mu_D + \frac{z_{\frac{\alpha}{2}}\sigma_D}{\sqrt{N}}. \tag{1.11}$$

The CI for the probability may be calculated identically using the vector X

$$P_L = \mu_X - \frac{z_{\frac{\alpha}{2}}\sigma_X}{\sqrt{N}} \tag{1.12}$$

$$P_U = \mu_X + \frac{z_{\frac{\alpha}{2}}\sigma_X}{\sqrt{N}}. \tag{1.13}$$

1.3.3 Markov Chain Model

Discrete-Time Markov Chains (DTMCs) are characterised by a discrete-time state space, where the state may take a single value at each time. This report focuses on two-state models, as shown by Figure 1.3.

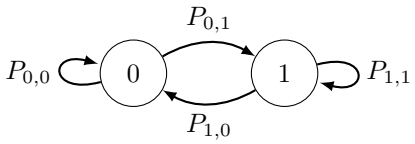


Figure 1.3: Two State DTMC Model

From a current state, we may remain or move to the alternate state with probabilities determined by their related transition matrix:

$$\pi = \begin{bmatrix} P_{0,0} & P_{0,1} \\ P_{1,0} & P_{1,1} \end{bmatrix}.$$

Continuous-Time Markov Chains (CTMCs) have frequently been used to understand availability, either through their related Poisson processes [11] or within a Monte-Carlo simulation (CTMC-MC) [4, 7, 8]. Additionally, Duard utilised a multi-element, multi-state Hidden Markov Model (HMM) [6] to forecast wait times. This report aims to reduce computational complexity through state reduction, time-discretisation, and analytical estimates derived from the transition matrix instead of inferred estimates from simulated data.

1.3.4 DTMC Model

We use a DTMC to model the accessibility of a site at a given time, $\mathbf{X} = \{X_1, X_2, \cdots, X_n\}$, where $X_t = 1$ indicates that there is an access window beginning at time t. This model assumes a constant probability of accessibility, $Pr(X_t = 1) = P$, and of transition between states

$$Pr(X_t = i | X_{t-1} = j) = P_{i,j}.$$

Probability of Instant Access

For $0.1 \leq P \leq 0.9$, we may construct a confidence interval for P as (P_L, P_U) [18, 19], where the limits are defined as

$$P_U = \frac{S + \frac{z_{\frac{\alpha}{2}}}{2} + \left[\frac{1}{2} + z_{\frac{\alpha}{2}}\sqrt{h\left(\hat{\theta}\right)\left\{S + \frac{1}{2} - \frac{\left(S+\frac{1}{2}\right)^2}{n} + \frac{z^2_{\frac{\alpha}{2}}h\left(\hat{\theta}\right)}{4}\right\}}\right]}{n + z^2_{\frac{\alpha}{2}}h\left(\hat{\theta}\right)} \tag{1.14}$$

$$P_L = \frac{S + \frac{z_{\frac{\alpha}{2}}}{2} - \left[\frac{1}{2} + z_{\frac{\alpha}{2}}\sqrt{h\left(\hat{\theta}\right)\left\{S - \frac{1}{2} - \frac{\left(S-\frac{1}{2}\right)^2}{n} + \frac{z^2_{\frac{\alpha}{2}}h\left(\hat{\theta}\right)}{4}\right\}}\right]}{n + z^2_{\frac{\alpha}{2}}h\left(\hat{\theta}\right)}, \tag{1.15}$$

where $S = \sum_{i=1}^{n} X_i$ denotes the number of observed access points, $z_{1-\frac{\alpha}{2}}$ is the $(1 - \frac{\alpha}{2})^{th}$ quantile of the standard normal distribution and

$$h(\theta) = 1 + \frac{2\theta}{1-\theta}\frac{1-(1-\theta^n)}{n(1-\theta)} \tag{1.16}$$

is a function of the product-moment correlation coefficient between adjacent states X_k and X_{k+1}, given by $\theta = \frac{P_{1,1}-P}{1-P}$ [18].

If P, or $1 - P$, is very small (here we consider 0.1 to be an appropriate threshold), then this method is not appropriate, and instead, a modified Anderson-Burstein interval should be utilised [20]. Since S is assumed to be Poisson-distributed with parameter nP, then we may write

$$P_U = \hat{P} + \left(P_{UI} - \hat{P}\right)\sqrt{1 + \frac{2\hat{\theta}}{1-\hat{\theta}}} \tag{1.17}$$

$$P_L = \hat{P} - \left(\hat{P} - P_{LI}\right)\sqrt{1 + \frac{2\hat{\theta}}{1-\hat{\theta}}}, \tag{1.18}$$

where (L, U) is the Pearson-Hartley confidence interval for the Poisson mean with parameter nP, and

$$P_{UI} = \frac{U}{n + d + \frac{1}{2}(U - S)} \tag{1.19}$$

$$P_{LI} = \frac{L}{n - \frac{1}{2}(S - 1 - L)} \tag{1.20}$$

are the Anderson-Burstein confidence limits with adjustment factor d [21, 22].

Expected Delay

The expected delay, λ, is given by

$$\lambda = P\lambda_1 + (1-P)\lambda_0 \tag{1.21}$$

$$= P \cdot 0 + (1-P)\lambda_0 \tag{1.22}$$

$$= (1-P)\lambda_0, \tag{1.23}$$

where λ_0 is calculated as

$$\lambda_0 = P_{0,0} \cdot (\lambda_0 + 1) + P_{0,1} \cdot 1 \tag{1.24}$$
$$\Rightarrow \lambda_0 = 1 + P_{0,0}\lambda_0 \tag{1.25}$$
$$\Rightarrow \lambda_0(1 - P_{0,0}) = 1 \tag{1.26}$$
$$\Rightarrow \lambda_0 = \frac{1}{1 - P_{0,0}} \tag{1.27}$$
$$\Rightarrow \lambda_0 = \frac{1}{P_{0,1}}. \tag{1.28}$$

The expected value for λ_0 is given by the sample mean of wait times from inaccessible states, $\hat{\lambda}_0$. To assess the variance of this parameter, we then consider the variance of its inverse $P_{0,1}$.

Confidence intervals for the transition matrix properties are outlined as [23]

$$P^U_{i,j} = p_{i,j} + z_{1-\frac{\alpha}{2}} \sqrt{\frac{p_{i,j}(1 - p_{i,j})}{f_i}} \tag{1.29}$$

$$P^L_{i,j} = p_{i,j} - z_{1-\frac{\alpha}{2}} \sqrt{\frac{p_{i,j}(1 - p_{i,j})}{f_i}} \tag{1.30}$$

where $f_i = \sum_{k=1}^{n} \mathbb{I}(X_k = i)$ is the number of observed values of state i.

We may then calculate the variance for $\lambda = \frac{1}{P_{0,1}}$ [24]

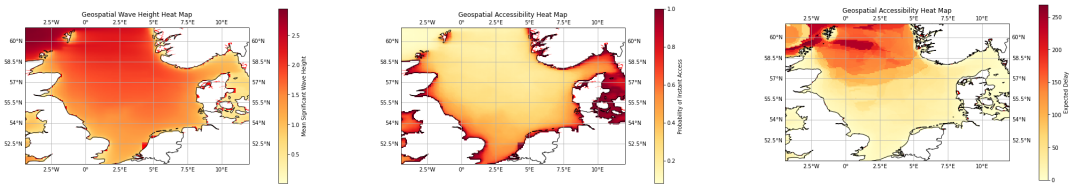
$$V\left(\frac{1}{P_{0,1}}\right) = \left(\frac{1}{\hat{P}_{0,1}}\right)^2 \left[\frac{V(1)}{1^2} + \frac{V(\hat{P}_{0,1})}{\hat{P}^2_{0,1}} - \frac{2C\left(1, \hat{P}_{0,1}\right)}{1 \cdot \hat{P}_{0,1}}\right] \tag{1.31}$$
$$= \frac{V(\hat{P}_{0,1})}{\hat{P}^4_{0,1}} \tag{1.32}$$
$$= \frac{\left(\frac{P^U_{0,1} - P^L_{0,1}}{2z_{1-\frac{\alpha}{2}}}\right)^2}{\hat{P}^4_{0,1}} \tag{1.33}$$
$$= \frac{(P^U_{0,1} - P^L_{0,1})^2}{4\hat{P}^4_{0,1} z^2_{1-\frac{\alpha}{2}}}. \tag{1.34}$$

The order of this model is also entirely linear but with a smaller factor, thus reducing the computation time from the empirical.

1.4 Results

1.4.1 Geospatial Analysis

An empirical analysis of these accessibility traits of various points in the North Sea was conducted using data gathered by the E.U. Copernicus Marine Service Information [25]. Exhaustive high-resolution points in a 0.08-degree grid across the North Sea were assessed to identify the probability of an expected wait time until an access window is available to carry out a 12-hour operation with a threshold safe wave height of 1.5m. The results of these are shown in Figure 1.4 The probability of instant access appears to be the inverse of mean significant wave height and thus provides limited additional information. However, whilst certain points seem undifferentiated in terms of wave height and access probability, they exhibit vastly different expected delay times. This indicates that this measure provides valuable information as to which sites may be difficult to access due to more fluctuating weather.



(a) Mean Significant Wave Height (b) Probability of Instant Access (c) Expected Delay

Figure 1.4: North Sea Geospatial Accessibility Heat Map

1.4.2 Goodness-of-fit Tests

The continuous-time models were assessed using likelihood, AIC and BIC scores, and goodness-of-fit tests, including Chi-Square and Kolmogorov-Smirnoff. The Weibull distribution performed well on each of these measures across multiple datasets, providing the highest likelihood and most significant goodness-of-fit test results for significant wave height. However, the AIC scores indicated that the gamma distribution was a higher-performing model in all three cases whilst providing equally significant fits.

Consider a Discrete-Time Markov Chain with starting state 0. At any given time step, the probability of achieving state 1 at the next stage is given by the transition matrix probability $\pi_{0,1}$. This may be equivalently considered as a Bernoulli experiment with probability of success $p = \pi_{0,1}$, and we may thus consider the runs of inaccessible states (continuous-times spent at state 0) to be distributed geometrically with parameter p. This assumed distribution is plotted against the observed data in Figure 1.5. A Chi-square goodness-of-fit test was conducted, giving a p-value less than 0.01, thus providing no evidence to reject this assumption. Hence, the Markov model is considered a good fit for this data. Additionally, a Monte Carlo simulated DTMC sequence of states using the fitted chain was compared to the observed data using both Chi-square and Kolmogorov-Smirnov goodness-of-fit tests, which also supported the model fit.

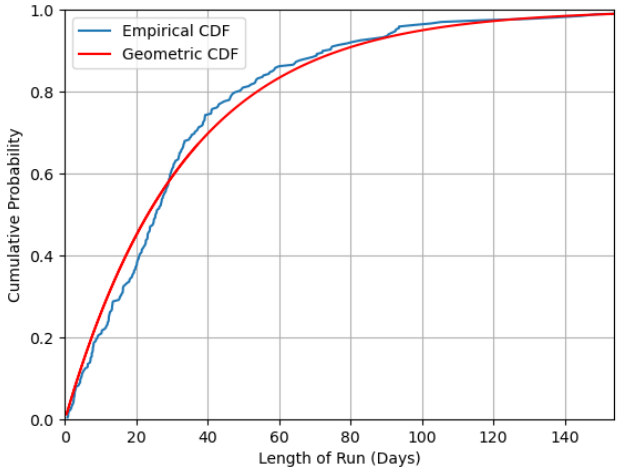


Figure 1.5: Cumulative Distribution of Inaccessibility Runs

1.4.3 Percentage Error

The efficacy of the models was tested against three use cases: the Barrow hourly data set [26, 27] used in the original paper [14], the Copernicus tri-hourly dataset [25] and the Firth of Forth half-hourly dataset. The percentage errors are outlined in Tables 1.1, 1.2 and 1.3, respectively.

Model	P_0	% Error	Expected Delay	% Error
Empirical	0.715	-	10.1	-
DTMC	0.715	0	10.3	1.79
MLE	0.759	6.11	10.9	8.41
Moments	0.759	7.04	14.2	41.0

Table 1.1: Barrow Data Set

Model	P_0	% Error	Expected Delay	% Error
Empirical	0.754	-	12.3	-
DTMC	0.754	0	12.3	0.730
MLE	0.745	1.19	11.3	8.13
Moments	0.748	0.768	11.1	9.76

Table 1.2: Copernicus Tri-hourly Data Set

Model	P_0	% Error	Expected Delay	% Error
Empirical	0.754	-	16.0	-
DTMC	0.754	0	14.7	7.95
MLE	0.728	3.51	15.1	5.66
Moments	0.727	3.27	17.6	9.87

Table 1.3: Firth of Forth Half-hourly Data Set

The DTMC performed the best on average across the three-models, however was outperformed by the MLE model on the Firth of Forth dataset. The Markov model is also more computationally efficient since it requires no distribution fitting and is of lower order than empirical estimates.

1.4.4 Confidence Intervals

Point estimates are accurate within 10% for the Empirical, DTMC and MLE models, although these percentage errors increase when we consider future values [Appendix 5.1.1]. This is due to a combination of year-to-year variation and differences in extreme values in a larger training set applied to a smaller test set. An attempt to reduce this variation by splitting data seasonally (monthly, triannually and biannually) was trialled, but the reduction in training data size led to more significant variation and subsequent errors. Thus, confidence intervals are essential to quantify the uncertainty associated with extension to future conditions. A 16-year dataset [28] was used to provide training (13-year) and test (3-year) data for forecasting. The confidence intervals and adjusted confidence intervals (with variation scaled for reduction in test date size) for these are given in Table 1.4.

H_{th} (m)	2	3	2	3	2	3
t_{req} (hours)	12	12	24	24	48	48
Observed Mean Delay	42	9.29	53.9	11.0	68.8	14.3
Forecasted DTMC 95% CI	(0,72.5)	(0.085, 8.71)	(0,160)	(2.19,20.0)	(0,160)	(2.19,20.0)
Forecasted DTMC 95% CI (adjusted)	(0,130)	(0, 13.0)	(0,187)	(0, 18.2)	(0,282)	(2.19,29.0)

Table 1.4: Confidence Intervals of Forecasted Expected Delay (Days)

1.5 Use in O&M Cost Prediction

To emulate the relevant parts of LCOE, these outputs have been used to calculate Operation & Maintenance Cost per Kw per year. For each type of repair, the failure/cost data outlined in Carrol et al (2016) [29] were used to estimate baseline values for the following

- Number of occurrences over specified period
- Probability of occurrence over specified period
- Associated required operation time
- Vessel required
- Associated threshold safe wave height and wind speed for operation
- Cost of repair

These were then combined to calculate the cost as

$$Cost = \sum_{m=1}^{M} \lambda_m \cdot C_m$$

and downtime

$$Downtime = \sum_{m=1}^{M} \lambda_m \cdot T_{op}(m) + T_{travel}(m, loc) + \mathbb{E}\left[T_{delay}\left(T_{op}(m) + T_{travel}(m, loc)\right)\right],$$

where m is the type of operation or maintenance required, λ_m is the number of expected times m must be carried out annually, $T_{op}(m)$ is the time required to carry out operation m,

$$T_{travel}(m, loc) = 2 \cdot \frac{\text{Distance between location and coastline}}{\text{Speed of vessel required for operation } m}$$

is the required travel time to the location and

$$\mathbb{E}\left[T_{delay}\left(T_{op}(m) + T_{travel}(m, loc)\right)\right]$$

is the expected delay time until a weather window the length of travel time and operation time occurs.

The time to travel to and from the asset is calculated based on its distance to the closest coastline and the speed of the relevant vessel. A relevant extension would allow calculation of distance to closest port.

This time and the time to carry out the operation are combined. This total vessel trip time is then fed into a Discrete Markov Chain Model, which calculates the expected time until a weather window is reached where this trip can be safely conducted. This safe window requires that the vessel's threshold wave height and wind speed are not exceeded for the entire duration. The Markov Chain Model is briefly described later in this section.

This downtime is summed with inactive power hours (where wind speed is below cut-in or above cut-out speed) and divided by total hours to calculate availability

$$Availability = \frac{\text{Number of operating hours} - \text{Downtime}}{8760}.$$

Power for each hour is calculated using the power curve for the NREL 12MW reference turbine (or variable input) with

$$P = \begin{cases} 0 & \text{if } v < v_{cut-in} \\ \frac{1}{2} A C_p \rho (v^3) \eta & \text{if } v_{cut-in} \leq v < v_{rated} \\ P_{rated} & \text{if } v_{rated} \leq v < v_{cut_o ut} \\ 0 & \text{if } v \geq v_{cut-out} \end{cases}.$$

The average energy yield (AEY) is then scaled to remove estimated downtime

$$AEY = \sum_{t=1}^{T} P_t \cdot H_t \cdot \frac{\text{Number of Operating Hours} - \text{Downtime}}{\text{Number of Operating Hours}}.$$

Finally, the estimated cost of Operation and Maintenance is calculated as

$$C/kw/yr = \frac{\text{Cost}}{\text{AEY}}.$$

1.6 Conclusions

This section aimed to implement computational methods for estimating key accessibility measures, including the probability of instantaneous access at a given point and the expected time until the next access window. Various methods were developed and tested, namely Discrete Markov Chains and Distribution-based approaches. All models produced estimates within a 10% error of the true expected delay and instant access probability. The Discrete Markov Chain performed best across large-scale datasets and exhibited a lower computational complexity than its distributional counterparts. This model also produced confidence intervals for forecasted future delays containing the true values across various tested scenarios. An abridged version of the functions code, as well as error tables, are provided in Appendix A. All functions, as well as an example sheet and the validation code, can be found via github.

Chapter 2

Wake Modelling

Abstract

Turbine wakes can significantly impact dynamic loads, energy yield and power losses [1]. Thus, it is important to understand the effect of wakes on the wind field to carry out accurate resource assessment and energy yield predictions. This chapter aims to outline a novel methodology for calculating the wake deficit both accurately and efficiently in the software Strathfarm by using an analytical Fourier series approximation for the Bastankhah Porte-Agel wake model [1].

In order to fully derive the model mechanics, Section 2.2 first outlines simplistic methods such as the Frandsen [30] and Jensen [31] models, which have been used to approximate the wake shape as a 'top-hat'. The basis for the Frandsen model is then altered in Section 2.3 to incorporate the *Gaussian profile* of the wake suggested by wind-tunnel testing [1]. This forms the Bastankhah Porte-Agel model [1]. This model provides increased accuracy, but due to its additional shape complexity requires higher computational power and/or time.

Section 2.3 describes how the Fourier series can be used to approximate this model to improve computational efficiency, whilst maintaining accuracy. Section 2.3.1 first discusses the current approximation carried out in Strathfarm: an adapted Fourier series, using 6-dimensional lookup tables to calculate the coefficients, with a high computational cost. Section 2.3.2-2.3.4 then derives and reformulates the Fourier series, evaluating it analytically to remove the requirement for lookup tables, therefore greatly improving the computational efficiency of the simulation. The efficacies of each method are then tested and compared in Section 2.4 through 9 wind farm simulations within the Strathfarm software. The results in Section 2.5 show a percentage difference of around 7%

2.1 Introduction

When assessing and modelling wind farm conditions, the effect of wakes on downwind turbines must be considered to understand resulting power losses due to the associated velocity deficits. Although it is common for simple analytical models such as the 'top-hat' formulation proposed by Jensen (1983) [31] to be used, wind tunnel testing and mathematical modelling have shown that improved efficacy can be achieved by Gaussian ('sombrero') velocity deficit models. However, due to increased model complexity, these incur a greater computational cost [1].

Strathfarm is a Simulink-based simulation tool developed by the University of Strathclyde. This software accurately models and simulates wind farm conditions, allowing fully flexible model creation [32]. Tools such as these wish to achieve high precision with reasonable computational speed. Specifically, Strathfarm is designed to be run on a standard PC in real-time, with medium fidelity [33]. To achieve the level of accuracy required, two-parameter Gaussian wake-modelling is currently used, however the current methodology fails to meet the desired efficiency.

This project aims to reformulate Strathfarm's current modelling approach to achieve a more computationally efficient simulation while maintaining high predictive accuracy.

2.2 Prior Models

The following sections will summarize prevalent methods used for wake modelling as they are defined within their defining papers. This is in order for the reader to be familiar with the nomenclature typically used for this type of modelling, however it may make the task of comparing models difficult, and requires many abbreviations and symbols to be used with some representing the same, or very similar information. A glossary is provided in Appendix B to prevent confusion.

2.2.1 The Jensen Model

The Jensen model [31] calculates the wake deficit through the conservation of momentum of a linearly expanding wake. Consider a turbine, i, at a distance x downstream of a linearly expanding wake resulting from a single turbine, j, as illustrated in Figure 1. The model assumes that the momentum through the wake at x metres behind the rotor is equal to the momentum of flow through an identically sized radius immediately behind turbine j, formally

$$v\pi r^2 = \pi r_0^2 v_0 + \pi(r^2 - r_0^2)u, \tag{2.1}$$

where u, v_0, and v refer to the unimpeded wind speed, wind speed immediately behind the rotor, and the wind speed at distance x behind the rotor respectfully, and r_0 and r refer to the initial wake radius and wake radius at x respectfully.

Since the wake expands linearly, we may write that

$$r = r_0 + \alpha x, \tag{2.2}$$

where α is some constant, and from classical theory, we may represent

$$v_0 = u(1 - 2a), \tag{2.3}$$

where a is the speed ratio. Combining these results, we may rewrite the wake deficit as

$$v\pi(r_0 + \alpha x)^2 = \pi u(r_0^2(1 - 2a) + (r_0 + \alpha x)^2 - r_0^2) \tag{2.4}$$

$$\Rightarrow v = u\left(1 - \frac{2ar_0^2}{(r_0 + \alpha x)^2}\right). \tag{2.5}$$

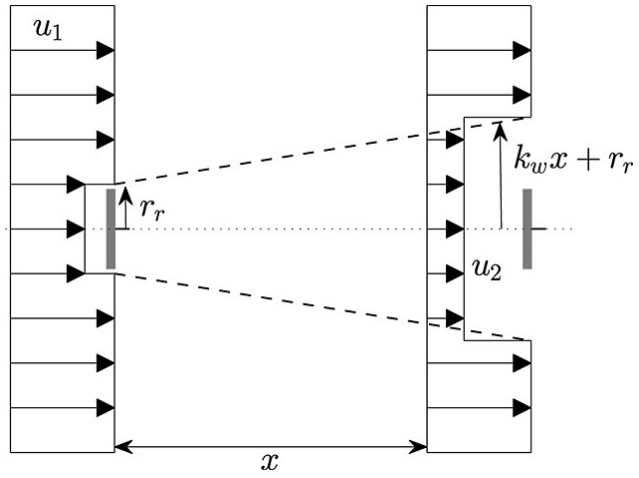


Figure 2.1: Jensen Wake Model [34]

Substituting $a = \frac{1}{2} - \frac{1}{2}\sqrt{1 - C_T}$, gives the wake deficit as

$$\delta v = \frac{v - u_\infty}{u_\infty} = \frac{1 - \sqrt{1 - C_T}}{\left(1 + \frac{\alpha x}{r_0}\right)^2}. \tag{2.6}$$

2.2.2 The Frandsen Model

The Frandsen model [35] instead considers a cylinder of air with a diameter equal to the wake diameter and surface area A_T [Figure 2.2].

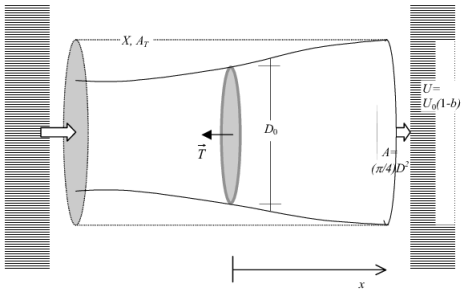


Figure 2.2: Cyclindrical Control Volume [35]

The momentum within this cylindrical flow volume is given by

$$\int_X \rho \frac{\delta \vec{U}}{\delta t} dX + \int_{A_T} \rho \vec{U}(\vec{U} d\vec{A}) = -\int_{A_T} \rho d\vec{A} + \int_X \rho \vec{g} dX + \vec{T} + \int_{A_T} \vec{\tau} dA. \tag{2.7}$$

The acceleration, pressure, gravity and shear terms are neglected to form the simplified equation

$$T = \int_A \rho U(U_0 - U) dA, \tag{2.8}$$

where A is the cross-sectional area, U_0 is the free stream velocity and U is the wake velocity. Assuming self-similarity of the wake, the wake velocity may be rewritten as

$$U = U_W(x) f(\bar{r}), \tag{2.9}$$

where U_W is the minimum wake flow speed, and $\bar{r} = \frac{r}{R}$ denotes the ratio of the distance from the centre of the wake to the wake radius. This may be substituted into equation 2.6 to give

$$T = \int_0^{2\pi} \int_0^{\infty} \rho r U_W f(\bar{r}) \left(U_0 - U_W f(\bar{r})\right) dr. \tag{2.10}$$

Hence

$$T \propto \rho R^2 \int_0^{\infty} \bar{r} U_W f(\bar{r} \left(U_0 - U_W f(\bar{r})\right) d\bar{r}. \tag{2.11}$$

Since this is dependent only on the minimum wake speed, U_W, it may be written that

$$T = C_{f1} \rho \pi R^2 U'_W (U_0 - U'_W), \tag{2.12}$$

where $U'_W = C_{f2} U_W$ is a 'characteristic' flow speed with C_{f1} and C_{f2} dependent on $\int_0^{\infty} f(\bar{r}) d\bar{r}$ and $\int_0^{\infty} f^2(\bar{r}) d\bar{r}$ respectively, and hence dependent on the shape of the wake profile. By the assumption of self-similarity, $C_{f1} = C_{f2} = 1$ and hence

$$T = \rho \pi R^2 U_W (U_0 - U_W) = \frac{\pi \rho}{4} D^2 U_W (U_0 - U_W). \tag{2.13}$$

Rotor thrust is given by

$$T = \frac{1}{2} \frac{\pi}{4} \rho D_0^2 U_0^2 C_T. \tag{2.14}$$

Frandsen defines the induction factor $a_F = 2a$, where a is the traditionally defined induction factor. Thus

$$C_T = a_F (2 - a_F) \Rightarrow a_F = 1 - \sqrt{1 - C_T} \tag{2.15}$$

giving the wake cross-sectional area immediately after expansion as

$$\frac{A_a}{A_0} = \frac{1 - \frac{1}{2}}{1 - a} \Rightarrow A_a = \beta A_0, \tag{2.16}$$

where

$$\beta = \frac{1 + \sqrt{1 - C_T}}{2\sqrt{1 - C_T}}. \tag{2.17}$$

Similarly, the diameter immediately after wake expansion may be written as

$$D_a = \sqrt{\beta} D_0.$$

Assuming immediate wake expansion, the flow speed may be calculated as

$$\frac{U}{U_0} = \frac{1}{2} \pm \frac{1}{2} \sqrt{1 - 2C_T \frac{A_0}{A}},$$

where the second term is additive exactly when $a_F < 0.5$. For small wake speed the approximation

$$T \approx \rho A U_0 (U_0 - U) \tag{2.18}$$

may be taken to produce a simplified form as

$$\frac{U}{U_0} \approx 1 - \frac{1}{2} C_T \frac{A_0}{A} \approx 1 - a_F \frac{A_0}{A}.$$

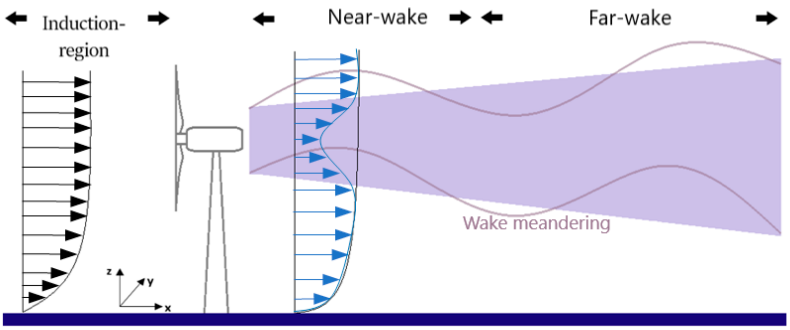


Figure 2.3: Gaussian Turbine Wake [36]

2.3 Current Model

2.3.1 The Bastankhah Porte-Agel (BPA) Model

Whilst the Fransen and Jensen models consider a 'top-hat' shaped wake, wind-tunnel testing data suggests a Gaussian distribution to be more representative. The BPA model [1] incorporates this Gaussian wake distribution, as shown in Figure 2.3
The derivation for this model begins as with the Frandsen model by considering the momentum through a cylindrical control volume [Figure 2.2], again neglecting the gravitational, shear, pressure and acceleration term to achieve

$$T = \int_A \rho U(U_0 - U) dA, \tag{2.19}$$

but diverges here by taking

$$U = U_W(x) f\left(e^{-\frac{r^2}{2\sigma^2}}\right) \tag{2.20}$$

that is assuming that the wake varies as a Gaussian distribution in the x direction centred at $(l, 0, z_h)$ where l is the lateral offset and z_h is the hub height.

$$f(e^{-\frac{r^2}{2\sigma^2}}) = \exp\left(-\frac{(z-z_0)^2}{2\sigma^2} + \frac{(y-y_0)^2}{2\sigma^2}\right) \tag{2.21}$$

$$= \exp\left(-\frac{(z-z_h)^2}{2\sigma^2} + \frac{(y)^2}{2\sigma^2}\right) \tag{2.22}$$

The wind speed within the wake is then given by

$$U_W(x) = U_\infty\left(1 - C(x) f\left(e^{-\frac{r^2}{2\sigma^2}}\right)\right), \tag{2.23}$$

where

$$C(x) = 1 - \sqrt{1 - \frac{C_T}{8\left(\frac{\sigma}{d_0}\right)^2}}. \tag{2.24}$$

Assuming a linearly increasing wake suggests a linearly increasing Gaussian standard deviation and thus

$$\frac{\sigma}{d_0} = k^* \frac{x}{d_0} + \varepsilon. \tag{2.25}$$

These results are combined to provide the wake deficit as

$$\frac{\Delta U}{U_\infty} = \frac{U_\infty - U}{U_\infty} \tag{2.26}$$

$$= \frac{U_\infty - U_\infty \left(1 - C(x)\right) f\left(e^{-\frac{r^2}{2\sigma^2}}\right)}{U_\infty} \tag{2.27}$$

$$= 1 - \left(1 - \left(1 - C(x)\right) f\left(e^{-\frac{r^2}{2\sigma^2}}\right)\right) \tag{2.28}$$

$$= \left(1 - C(x)\right) e^{-\frac{r^2}{2\sigma^2}} \exp\left(-\frac{(z - z_0)^2}{2\sigma^2} + \frac{y^2}{2\sigma^2}\right) \tag{2.29}$$

$$= \left(1 - \sqrt{1 - \frac{C_T}{8\left(\frac{k^* x}{d_0} + \varepsilon\right)^2}}\right) \exp\left\{-\frac{(z - z_h)^2}{2\sigma^2} + \frac{y^2}{2\sigma^2}\right\} \tag{2.30}$$

$$= \left(1 - \sqrt{1 - \frac{C_T}{8\left(\frac{k^* x}{d_0} + \varepsilon\right)^2}}\right) \exp\left\{-\frac{r^2}{2\sigma^2}\left(\frac{(z - z_h)^2}{r^2} + \frac{y^2}{r^2}\right)\right\} \tag{2.31}$$

$$= \left(1 - \sqrt{1 - \frac{C_T}{8\left(\frac{k^* x}{d_0} + \varepsilon\right)^2}}\right) \exp\left\{-\frac{2r^2}{\sigma^2}\left(\frac{(z - z_h)^2}{d_0^2} + \frac{y^2}{d_0^2}\right)\right\} \tag{2.32}$$

$$= \left(1 - \sqrt{1 - \frac{C_T}{8\left(\frac{k^* x}{d_0} + \varepsilon\right)^2}}\right) \exp\left\{-\frac{1}{2\left(\frac{k^* x}{d_0} + \varepsilon\right)^2}\left(\left(\frac{z - z_h}{d_0}\right)^2 + \left(\frac{\bar{r}}{d_0}\right)^2\right)\right\}, \tag{2.33}$$

where $\varepsilon = 0.25\sqrt{\beta}$ and

$$k^* = 0.3837I + 0.003678, \tag{2.34}$$

with I denoting the local streamwise turbulence intensity immediately upwind of the rotor centre.

2.3.2 Reformulation

Considering now a non-zero offset in both the x *and* y directions resulting in a new centre of (x_1, y_1), we may demonstrate the Gaussian distribution of the wake as a function of the lateral and longitudinal position x and y as

$$f(x, y) = A \exp\left(-\frac{(x - x_0)^2}{2\sigma^2} + \frac{(y - y_0)^2}{2\sigma^2}\right).$$

Assuming an original position of $(r\cos(\theta), r\sin(\theta))$ and an offset of $(0, l)$, the distribution is then given by

$$f(x, y) = A \exp\left(-\frac{(r\cos(\theta) - 0)^2}{2\sigma^2} + \frac{(r\sin(\theta) - l)^2}{2\sigma^2 y}\right) \tag{2.35}$$

$$= A \exp\left(-\frac{r^2\cos^2(\theta)}{2\sigma^2} - \frac{r^2\sin^2(\theta) - 2rl\sin(\theta) + l^2}{2\sigma^2}\right) \tag{2.36}$$

$$= A \exp\left(\frac{r^2 - 2rl\sin(\theta) + l^2}{2\sigma^2}\right). \tag{2.37}$$

Strathfarm calculates the wake deficit due to the interaction of the Gaussian wake with the affected turbines' rotor swept area [Figure 2.4]

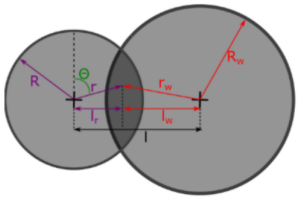


Figure 2.4: Interaction of Gaussian Wake Across Turbine's Swept Area [37]

as

$$\delta v(r,\theta) = -\frac{(c+1)V_m}{R^{c+1}} \int_0^R \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{l^2-2lr\sin(\theta)+r^2}{2\sigma^2}} r^c dr, \tag{2.38}$$

where V_m is the mean incoming wind speed, c is an adjustable weighting coefficient. We can see that the function varies only with regards to θ. We may now represent this function in terms of its Fourier series

$$\delta v(r,\theta) \sim \mathcal{F}(\theta) = a_0 + \sum_{n=1}^{\infty} a_n \cos\left(\frac{2\pi n\theta}{P}\right) + \sum_{n=1}^{\infty} b_n \sin\left(\frac{2\pi n\theta}{P}\right),$$

with

$$a_0 = \frac{1}{p}\int_P \delta v(\theta) d\theta \tag{2.39}$$

$$a_n = \frac{2}{p}\int_P \delta v(\theta) \cos\left(2\pi\frac{n}{p}\theta\right) d\theta \tag{2.40}$$

$$b_n = \frac{2}{p}\int_P \delta v(\theta) \sin\left(2\pi\frac{n}{p}\theta\right) d\theta. \tag{2.41}$$

Since the function is 2π periodic, and we can specify $\delta v(\theta)$ this may be rewritten as

$$\mathcal{F}(\theta) = a_0 + \sum_{n=1}^{\infty} a_n \cos(n\theta) + \sum_{n=1}^{\infty} b_n \sin(n\theta),$$

with

$$a_0 = \frac{1}{p}\int_P -\frac{(c+1)V_m}{R^{c+1}} \int_0^R \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{l^2-2lr\sin(\theta)+r^2}{2\sigma^2}} r^c dr d\theta \tag{2.42}$$

$$a_n = \frac{2}{p}\int_P -\frac{(c+1)V_m}{R^{c+1}} \int_0^R \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{l^2-2lr\sin(\theta)+r^2}{2\sigma^2}} r^c dr \cos(n\theta)) d\theta \tag{2.43}$$

$$b_n = \frac{2}{p}\int_P -\frac{(c+1)V_m}{R^{c+1}} \int_0^R \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{l^2-2lr\sin(\theta)+r^2}{2\sigma^2}} r^c dr \sin(n\theta) d\theta. \tag{2.44}$$

Expanding further, we have for $l > 0$

$$a_n = \frac{(c+1)V_m}{\pi R^{c+1}} \int_0^{2\pi} \int_0^R f(x,y) r^c dr \cos(n\theta) d\theta \tag{2.45}$$

$$= \frac{(c+1)V_m}{\pi R^{c+1}} \int_0^R \int_0^{2\pi} \frac{1}{2\pi\sigma} \exp\left(-\frac{r^2 - 2rl\sin(\theta) + l^2}{2\sigma^2}\right) \cos(n\theta) d\theta r^c dr \tag{2.46}$$

$$= \frac{(c+1)V_m}{2\pi^2\sigma R^{c+1}} \int_0^R \int_0^{2\pi} \exp\left(-\frac{lr(1 - sin(\theta)l^2}{\sigma^2}\right) \cos(n\theta) d\theta r^c \exp\left(-\frac{r^2 - 2lr}{2\sigma^2}\right) dr \tag{2.47}$$

$$= \frac{(c+1)V_m}{2\pi^2\sigma R^{c+1}} \exp\left(-\frac{-l^2}{2\sigma^2}\right) \int_0^R \int^2 \pi_0 \exp\left(-\delta\bar{r}(1 - \sin(\theta)\right) \cos(n\theta) d\theta \exp\left(-\frac{lr(1 - sin(\theta)l^2}{\sigma^2}\right) r^c \exp\left(-\frac{r^2 - 2lr}{2\sigma^2}\right) dr \tag{2.48}$$

$$= \frac{(c+1)V_m}{2\pi^2\sigma} g(\alpha, \beta), \tag{2.49}$$

where

$$g(\alpha,\beta) = \int_0^1 \bar{r}^c h(\alpha\beta\bar{r}) \exp\left(-\frac{-(\beta\bar{r} - \alpha)^2}{2}\right) d\bar{r}$$

and

$$h(\delta\bar{r}) = \int_0^{2\pi} \exp\left(-\delta\bar{r}(1 - \sin(\theta))\right) \cos(n\theta) d\theta,$$

with $\alpha = \frac{l}{\theta}$ and $\beta = \frac{R}{\theta}$.

Similarly,

$$b_n = \frac{(c+1)V_m}{2\pi^2\sigma} v(\alpha, \beta),$$

where

$$v(\alpha,\beta) = \int_0^1 \bar{r}^c u(\alpha\beta\bar{r}) \exp\left(-\frac{\beta\bar{r} - \alpha)^2}{2}\right) d\bar{r} \tag{2.50}$$

$$u(\delta\bar{r}) = \int_0^{2\pi} \exp\left(-\delta\bar{r}(1 - \sin(\theta))\right) \sin(n\theta) d\theta \tag{2.51}$$

For $l < 0$,

$$a_n = \frac{(c+1)V_m}{\pi R^{c+1}} \int_0^{2\pi} \int_0^R \frac{1}{2\pi\sigma} \exp\left(-\frac{l^2 - 2lr\sin(\theta) + r^2}{2\sigma^2}\right) r^c dr \cos(n\theta) d\theta \tag{2.52}$$

$$= \frac{(c+1)V_m}{\pi R^{c+1}} \int_{-\pi}^{\pi} \int_0^R \frac{1}{2\pi\sigma} \exp\left(-\frac{-\bar{l}^2 - \bar{l}r\sin(\phi) + r^2}{2\sigma^2}\right) r^c dr (-1)^n a \cos(n\phi) d\phi \tag{2.53}$$

$$= (-1)^n a_n|_{l>0}, \tag{2.54}$$

where $\bar{l} = -l$ and $\phi = \theta - \pi$. Similarly,

$$b_n|_{l<0} = (-1)^n b_n|_{l>0}.$$

2.3.3 Current Implementation in StrathFarm

Strathfarm currently uses a 4-dimensional lookup table based on a 6th-order polyfit of this Gaussian wake deficit. The inputs to the lookup tables are the distance between the turbines, upwind turbine CT, the lateral position of the wake compared to the centre of the downwind turbine, and the wind field's turbulence

intensity. Combining this with the reformulation in Section 2.3.2, the wake's change on the wind field may be considered in terms of the following parameters

$$a_{nP} = -\frac{c+1}{\pi R^{c+1}} V_m \int_0^{2\pi} f(r_w, \theta) \cos(n\theta)\, d\theta \tag{2.55}$$

$$b_{nP} = -\frac{c+1}{\pi R^{c+1}} V_m \int_0^{2\pi} f(r_w, \theta) \sin(n\theta)\, d\theta \tag{2.56}$$

$$\tag{2.57}$$

where a_nP and b_nP are Fourier coefficients, c is a weighting coefficient, θ is the azimuthal angle of rotation, r_w is the radial distance from the wake centre to an arbitrary point in the swept area of the rotor, and,

$$f(r_w, \theta) = \int \left(K_3 \left(l^2 - 2lr\sin(\theta) + r^2 \right)^3 + K_2 \left(l^2 - 2lr\sin(\theta) + r^2 \right)^2 + K_1 \left(l^2 - 2lr\sin(\theta) + r^2 \right) + K_0 \right) r^c dr,$$

where $K_i,\ i \in \{0, 1, 2, 3\}$ are functions of C_T, the turbulence intensity, distance downstream between turbines and lateral distance between the centre of the wind turbine's rotor and the centre of the wake, l. The term $l^2 - 2lr\sin(\theta) + r^2$ accounts for the offset in the Gaussian wake from the centre of the turbine requiring an offset in the y direction. This term can be calculated by considering the first principles of integrating in a polar reference frame with a y direction offset of l. We man non-dimensionalise by using $\bar{r} = \frac{r}{R}$ and $\bar{l} = \frac{l}{R}$ to give

$$f(r_w, \theta) = \int_0^1 \left(\bar{K}_3 \left(\bar{l}^2 - 2\bar{l}\bar{r}\sin(\theta) + \bar{r}^2 \right)^3 + \bar{K}_2 \left(\bar{l}^2 - 2\bar{l}\bar{r}\sin(\theta) + \bar{r}^2 \right) + \bar{K}_1 \left(\bar{l}^2 - 2\bar{l}\bar{r}\sin(\theta) + \bar{r}^2 \right) + \bar{K}_0 \right) \bar{r}^c d\bar{r},$$

where $\bar{K}_n = K_n R^{2n+c}$. Thus, the Fourier coefficients may be found as

$$a_nP = -\frac{c+1}{\pi} V_m \int_0^{2\pi} (A + B + C + D)\cos(nP) d\theta \tag{2.58}$$

$$b_nP = -\frac{c+1}{\pi} V_m \int_0^{2\pi} (A + B + C + D)\sin(nP) d\theta \tag{2.59}$$

where

$$A = \frac{\bar{K}_3\bar{l}^6}{c+1} - \frac{6\bar{K}_3\bar{l}^5\sin\theta}{c+2} - \frac{6\bar{K}_3\bar{l}^4\cos(2\theta)}{c+3} - \frac{9\bar{K}_3\bar{l}^4}{c+3} - \tag{2.60}$$

$$\frac{18\bar{K}_3\bar{l}^3\sin\theta}{c+4} + \frac{2\bar{K}_3\bar{l}^3\sin(3\theta)}{c+4} - \frac{6\bar{K}_3\bar{l}^2\cos(2\theta)}{c+5} + \frac{9\bar{K}_3\bar{l}^2}{c+5} - \frac{6\bar{K}_3\bar{l}\sin\theta}{c+6} + \frac{\bar{K}_3}{c+7} \tag{2.61}$$

$$B = \frac{\bar{K}_2\bar{l}^4}{c+1} - \frac{4\bar{K}_2\bar{l}^3\sin\theta}{c+2} + \frac{4\bar{K}_2\bar{l}^2\sin^2(\theta)}{c+3} + \frac{2\bar{K}_2\bar{l}^2}{c+3} - \frac{4\bar{K}_2\bar{l}\sin\theta}{c+4} + \frac{\bar{K}_2}{c+5} \tag{2.62}$$

$$C = \frac{\bar{K}_1\bar{l}^2}{c+1} - \frac{2\bar{K}_1\bar{l}\sin\theta}{c+2} + \frac{\bar{K}_1}{c+3} \tag{2.63}$$

$$D = \frac{\bar{K}_0}{c+1}. \tag{2.64}$$

Due to the additive nature of the polynomial, each integral term may be considered separately. The non-zero Fourier coefficients are given as

$$a_{0P} = -(c+1)V_m\left(\frac{2\bar{K}_3\bar{l}^6}{c+1} + \frac{18\bar{K}_3\bar{l}^4}{c+3} + \frac{18\bar{K}_3\bar{l}^2}{c+5} + \frac{\bar{K}_3}{c+7} + \frac{2\bar{K}_2\bar{l}^4}{c+1} + \frac{2\bar{K}_2}{c+5} + \frac{8\bar{K}_2\bar{l}^2}{c+3} + \frac{2\bar{K}_1\bar{l}^2}{c+1} + \frac{2\bar{K}_1}{c+3} + \frac{2\bar{K}_0}{c+1}\right) \tag{2.65}$$

$$b_{1P} = -(c+1)V_m\left(-\frac{6\bar{K}_3\bar{l}^5}{c+2} - \frac{18\bar{K}_3\bar{l}^3}{c+4} - \frac{6\bar{K}_3\bar{l}}{c+6} + \frac{4\bar{K}_2\bar{l}^3}{c+2} - \frac{4\bar{K}_2\bar{l}}{c+4} - \frac{2\bar{K}_1\bar{l}}{c+2}\right) \tag{2.66}$$

$$a_{2P} = -(c+1)V_m\left(-\frac{6\bar{K}_3\bar{l}^4}{c+3} - \frac{6\bar{K}_3\bar{l}^2}{c+5} - \frac{2\bar{K}_2\bar{l}^2}{c+3}\right) \tag{2.67}$$

$$b_{3P} = -(c+1)\left(\frac{2\bar{K}_3\bar{l}^3}{c+4}\right). \tag{2.68}$$

Hence, an addition δv can be made to the wind speed local to the rotor where,

$$\delta v = \frac{a_{0P}}{2} + b_{1P}\sin(\theta) + a_{2P}\cos(2\theta) + b_{3P}\sin(3\theta). \tag{2.69}$$

In StrathFarm, lookup tables are used to find K values, which are then used to calculate the values of a_{NP} and b_{NP} [37].

2.3.4 Reformulation

As previously specified, the Fourier coefficient for the wake deficit equation a_n may be calculated as

$$a_n = \frac{(c+1)V_m}{\pi}\int_0^{2\pi}\exp\left\{-\frac{\bar{l}^2}{2K}\right\}\int_0^1 \exp\left\{\frac{2\bar{l}\bar{r}\sin(\theta) - \bar{r}^2}{2K}\right\}\bar{r}^c d\bar{r} f(\theta)\cos(n\theta)d\theta \tag{2.70}$$

$$= \frac{(c+1)V_m}{\pi}\exp\left\{-\frac{\bar{l}^2}{2K}\right\}\int_0^1\int_0^{2\pi}\exp\left\{\frac{\bar{l}\bar{r}\sin(\theta)}{K}\right\}\cos(n\theta)d\theta\exp\left\{-\frac{\bar{r}^2}{2K}\right\}\bar{r}^c d\bar{r} \tag{2.71}$$

The integral

$$I = \int_0^{2\pi}\exp\left\{\frac{\bar{l}\bar{r}\sin(\theta)}{K}\right\}\cos(n\theta)d\theta \tag{2.72}$$

cannot be analytically evaluated but may be numerically approximated by a 6th-order polynomial model of the form

$$I_A = \sum_{i=0}^{6} A_{n,i}\cdot\left(\frac{\bar{l}\bar{r}}{K}\right)^i. \tag{2.73}$$

Note that several models were fitted, including lower- and higher-order polynomials and probabilistic distributions. The 6th-order polynomial produced the lowest residual sum and provided a simple and more efficient model, however, when accounting for flexibility and model complexity an exponential model may be considered more suitable, this model is derived in Appendix A. The results of this model and its implications

for mode Hence, we may evaluate a_n as

$$a_n = \frac{(c+1)V_m}{\pi} \exp\left\{-\frac{\bar{l}^2}{2K}\right\} \int_0^1 \left(\sum_{i=0}^{6} A_{n,i} \cdot \left(\frac{\bar{l}\bar{r}}{K}\right)^i\right) \exp\left\{-\frac{\bar{r}^2}{2K}\right\} \bar{r}^c d\bar{r} \tag{2.74}$$

$$= \frac{(c+1)V_m}{\pi} \exp\left\{-\frac{\bar{l}^2}{2K}\right\} \left[\sum_{i=0}^{6} \frac{A_{n,i}\bar{l}^i}{K^i} \int_0^1 \exp\left\{-\frac{\bar{r}^2}{2K}\right\} \bar{r}^{c+i} d\bar{r}\right] \tag{2.75}$$

$$= \frac{(c+1)V_m}{\pi} \exp\left\{-\frac{\bar{l}^2}{2K}\right\} \left[\sum_{i=0}^{6} \frac{A_{n,i}\bar{l}^i}{K^i} \left(-K^{\frac{c+i+1}{2}} \cdot 2^{\frac{c+i-1}{2}} \cdot \gamma\left(\frac{c+i+1}{2}, \frac{1}{2K}\right)\right)\right] \tag{2.76}$$

$$= \frac{(c+1)V_m}{\pi} \exp\left\{-\frac{\bar{l}^2}{2K}\right\} \left[\sum_{i=0}^{6} -2^{\frac{c+i-1}{2}} A_{n,i}\bar{l}^i K^{\frac{c+1-i}{2}} \gamma\left(\frac{c+i+1}{2}, \frac{1}{2K}\right)\right] \tag{2.77}$$

Similarly,

$$I_B = \int_0^{2\pi} \exp\left\{\frac{\bar{l}\bar{r}}{K}\right\} \sin(n\theta) d\theta \approx \sum_{i=0}^{6} A_{n,i} \cdot \left(\frac{\bar{l}\bar{r}}{K}\right)^i \tag{2.78}$$

and thus b_n may be evaluated as

$$b_n = \frac{(c+1)V_m}{\pi} \exp\left\{-\frac{\bar{l}^2}{2K}\right\} \left[\sum_{i=0}^{6} -2^{\frac{c+i-1}{2}} B_{n,i}\bar{l}^i K^{\frac{c+1-i}{2}} \gamma\left(\frac{c+i+1}{2}, \frac{1}{2K}\right)\right] \tag{2.79}$$

It is provable that

$$a_n = 0 \ \forall n = 2m+1, \ m \in \mathbb{Z}$$

and

$$b_n = 0 \ \forall n = 2m, \ m \in \mathbb{Z},$$

i.e. a_n is zero for all odd values of n, and b_n is zero for all even values of n. This proof is given in the appendix. As well as this, the nature of the series a_n and b_n are diminishing sinusoidal approaching zero in such a way that we may take

$$a_n \approx b_n \approx 0, \ \forall n > 3$$

and estimate the entire system in terms of only a_0, b_1, a_2 and b_3. This aligns with the current approximations carried out by the existing Strathfarm software, which ignores all other coefficients. Thus, efficacy is not harmed by this simplifying assumption.

2.4 Implementation and Testing

The derived polynomial function was implemented into Strathfarm as shown in Figure 2.5, using Simulink block architecture.

A farm layout of 9 turbines in a 3-by-3 grid, as demonstrated by Figure 2.6, was then constructed and simulated for each of the following wind conditions [Table 2.1].

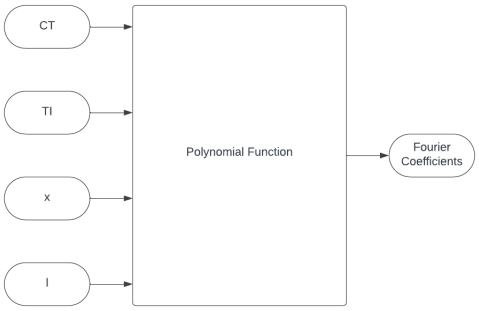


Figure 2.5: Strathfarm Implementation

Simulation	Mean Wind Speed (μ) [ms^{-1}]	Turbulence Intensity (TI) [%]
1	9	5
2	9	12
3	9	20
4	11	5
5	11	12
6	11	20
7	15	5
8	15	12
9	15	20

Table 2.1: Simulated Wind Conditions

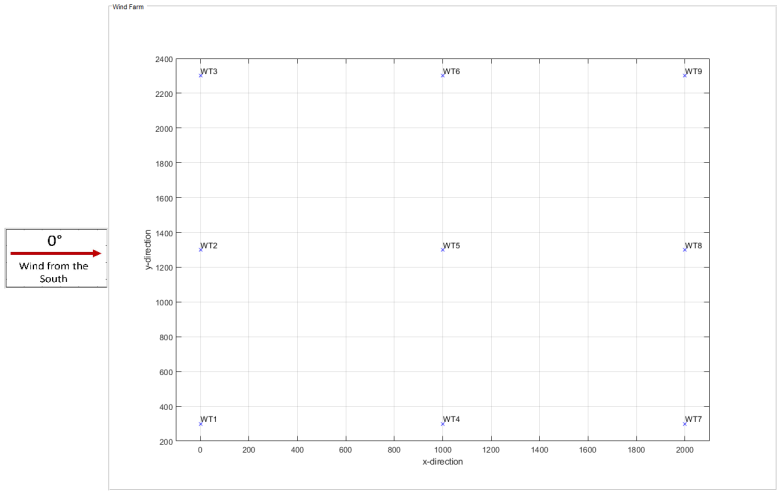


Figure 2.6: Simulated Farm Layout

2.5 Results

The Mean Percentage Error was measured across the 9 simulations, with an average of 7.1%, within the aimed 10% limit. Additionally, the Product Moment Correlation Coefficient (PMCC) between the original Strathfarm Lookup Table method and the proposed model was used to assess linearity. This allows us to limit errors to scaling issues rather than model misrepresentation. All models displayed a PMCC greater than 0.98, suggesting strong linearity and model agreement. This was further supported by low CUSUM test statistics, with average line ratios of 50.1:49.9 [Figure 2.7], as well as Chi-square GOF tests for each

simulation, which provided no evidence against linearity. The result linearity for simulation 9 (most complex conditions) is demonstrated visually by Figure 2.7.

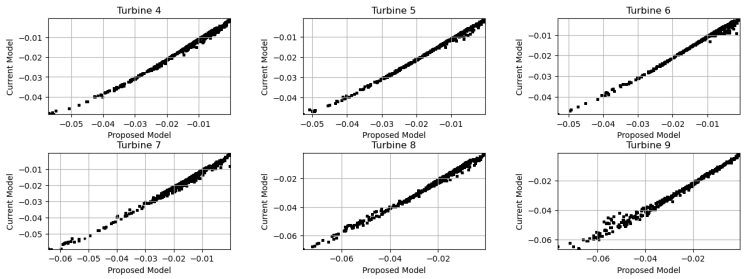


Figure 2.7: Simulation 9 Model Comparison

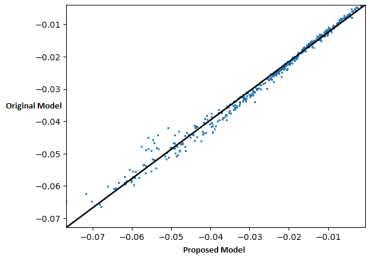


Figure 2.8: OLS Regression CUSUM

2.6 Limitations

There is still a relatively high percentage error of 7.1%. With the high-linearity between the two models. This error is likely due to a miscalculation of a constant scaling factor. Further model tuning should be conducted to assess whether there should be an additional or adjusted constant within the model. In addition, the simulations were only run on a single simulated set-up of a 9-turbine farm in a regular grid with a single wind direction perpendicular to turbine rows. The model should be further validated on various farm setups with non-regularized grids, larger sizes, and varying wind input directions.

2.7 Conclusions

A formula for Gaussian wake deficit based on a Fourier approximation of the Bastankhah Porte-Agel model was derived and implemented in Strathfarm to replace the existing look-up table framework. Simulations were conducted, showing greatly improved computational efficiency and errors of less than 10%. Due to the strong linearity between these models, these errors are likely due to poor estimation of a scaling parameter rather than an error in the integral model. Further work would yield an investigation into better parameter estimation and broader testing on different farm sizes and wind directions.

Bibliography

[1] M. Bastankhah and F. Portao-Agel. "A new analytical model for wind-turbine wakes". In: *Renewable Energy* 70 (2014), pp. 116–123.

[2] H. Zhang et al. "Uncertain accessibility estimation method for offshore wind farm based on multi-step probabilistic wave forecasting". In: *IET Renewable Power Generation* 15 (13 Oct. 2021), pp. 2944–2955. ISSN: 17521424. DOI: 10.1049/rpg2.12227.

[3] Y. Dalgic et al. "Advanced logistics planning for offshore wind farm operation and maintenance activities". In: *Ocean Engineering* 101 (June 2015), pp. 211–226. ISSN: 00298018. DOI: 10.1016/j.oceaneng.2015.04.040.

[4] M. Centeno-Telleria, J.I. Aizpurua, and M. Penalba. "Impact of accessibility on O&M of floating offshore wind turbines: Sensitivity of the deployment site". In: CRC Press/Balkema, 2023, pp. 847–855. ISBN: 9781032420035. DOI: 10.1201/9781003360773-94.

[5] J. Philips, C. Morgan, and J. Jacquemin. "Evaluating O&M strategies for offshore wind farms through simulation-the impact of wave climatology". In: *Proceedings of the OWEMES*. 2006, pp. 20–22.

[6] F. Douard, C. Domecq, and W. Lair. "A Probabilistic Approach to Introduce Risk Measurement Indicators to an Offshore Wind Project Evaluation – Improvement to an Existing Tool Ecume". In: *Energy Procedia* 24 (2012). Selected papers from Deep Sea Offshore Wind R&D Conference, Trondheim, Norway, 19-20 January 2012, pp. 255–262. ISSN: 1876-6102. DOI: https://doi.org/10.1016/j.egypro.2012.06.107. URL: https://www.sciencedirect.com/science/article/pii/S1876610212011472.

[7] I. Dinwoodie and D. McMillan. "Operational strategies for offshore wind turbines to mitigate failure rateuncertainty on operational costs and revenue". In: *Renewable Power Generation 8* (2014), pp. 359–366.

[8] G. Rinaldi et al. "Incorporating stochastic operation and maintenance models into the techno-economic analysis of floating offshore wind farms". In: *Applied Energy 301, 117420.* (2021).

[9] P. Papadopoulos, D.W. Coit, and A.A. Ezzat. "Seizing Opportunity: Maintenance Optimization in Offshore Wind Farms Considering Accessibility, Production, and Crew Dispatch". In: *IEEE Transactions on Sustainable Energy* 13 (1 Jan. 2022), pp. 111–121. ISSN: 19493037. DOI: 10.1109/TSTE.2021.3104982.

[10] I. Dinwoodie, F. Quail, and D. McMillan. "Analysis of offshore wind turbine operation and maintenance using a novel time domain meteo-ocean modeling approach". In: *Turbo Expo: Power for Land, Sea, and Air*. Vol. 44724. American Society of Mechanical Engineers. 2012, pp. 847–857.

[11] LL. Huang et al. "A Markov-Chain-Based Availability Model of Offshore Wind Turbine Considering Accessibility Problems". In: *IEEE Transactions on Sustainable Energy* 8 (4 Oct. 2017), pp. 1592–1600. ISSN: 19493029. DOI: 10.1109/TSTE.2017.2695661.

[12] F.A.R Abdullah, N.S. Ningsih, and T. M. Al-Khan. "Significant wave height forecasting using long short-term memory neural network in Indonesian waters". In: *Journal of Ocean Engineering and Marine Energy* 8 (2022), pp. 183–192. ISSN: 21986452. DOI: 10.1007/s40722-022-00224-3.

[13] J.B. Feuchtwang. "The offshore access problem and turbine availability-probabilistic modelling of expected delays to repairs". In: *European Offshore Wind Conference 2009, 2009-09-14 - 2009-09-16. (https://doi.org/10.1002/we.1539)* (2009).

[14] J.B. Feuchtwang and D.G. Infield. "Offshore wind turbine maintenance access: A closed-form probabilistic method for calculating delays caused by sea-state". In: *Wind Energy* 16 (7 2013), pp. 1049–1066. ISSN: 10991824. DOI: 10.1002/we.1539.

[15] J.B. Feuchtwang and D.G. Infield. *Estimating the cost of offshore maintenance and the benefit from condition monitoring*. 2011.

[16] J.B. Feuchtwang. "The offshore access problem and turbine availability-probabilistic modelling of expected delays to repairs, Figure 2". In: *European Offshore Wind Conference 2009, 2009-09-14 - 2009-09-16. (https://doi.org/10.1002/we.1539)* (2009).

[17] G.W. Cran. *Moment Estimators for the 3-Parameter Weibull Distribution*, p. 60.

[18] E.J. Bedrick and J. Aragon. *Approximate Confidence Intervals for the Parameters of a Stationary Binary Markov Chain*. 1989, pp. 437–448.

[19] E.L. Crow. "Approximate confidence intervals for a proportion from Markov dependent trials". In: *Communications in Statistics-Simulation and Computation* 8.1 (1979), pp. 1–24.

[20] E.L. Crow and M.J. Miles. "Validation of estimators of a proportion from Markov dependent trials". In: *Communications in Statistics-Simulation and Computation* 8.1 (1979), pp. 25–52.

[21] T.W. Anderson and H. Burstein. "Approximating the upper binomial confidence limit". In: *Journal of the American Statistical Association* 62.319 (1967), pp. 857–861.

[22] T.W. Anderson and H. Burstein. "Approximating the lower binomial confidence limit". In: *Journal of the American Statistical Association* 63.324 (1968), pp. 1413–1415.

[23] L. Wanand et al. "Confidence intervals for Markov chain transition probabilities based on next generation sequencing reads data". In: *Quantitative Biology* 8 (2 June 2020), pp. 143–154. ISSN: 20954697. DOI: 10.1007/s40484-020-0200-y.

[24] K.M. Wolter. *Introduction to Variance Estimation*. 2006.

[25] E.U. Copernicus Marine Service Information (CMEMS). *Global Ocean Waves Analysis and Forecast*. Tech. rep. Marine Data Store (MDS), 2023. URL: https://data.marine.copernicus.eu/product/GLOBAL_ANALYSISFORECAST_WAV_001_027/description.

[26] *BERR offshore wind farm capital grants scheme—Barrow offshore wind farm 1st annual report July 2006 – June 2007*. 2007. URL: http://webarchive.nationalarchives.gov.uk/+/http://www.berr.gov.uk/%EF%AC%81les/%EF%AC%81le50163.pdf.

[27] *BERR offshore wind farm capital grants scheme—Barrow offshore wind farm 2nd annual report July 2007 – June 2008*. 2008. URL: http://www.decc.gov.uk/assets/decc/what%20we%20do/a%%2020low%20carbon%20uk/1_20091120130307_e_@@_barrowannualreportjuly2007tojune2008%EF%AC%81nal.pdf.

[28] Government of Ireland. "Weather Buoy Network". In: *Marine Institute* (Sept. 14, 2017). URL: https://data.gov.ie/publisher/marine-institute.

[29] James Carroll, Alasdair McDonald, and David McMillan. "Failure rate, repair time and unscheduled O&M cost analysis of offshore wind turbines". In: *Wind energy* 19.6 (2016), pp. 1107–1119.

[30] S. Frandsen. *On the wind speed reduction in the center of large clusters of wind turbines*. 1992.

[31] N.O. Jensen. *A Note on Wind Turbine Interaction. Technical report. Ris-M-2411*.

[32] M.D. Cole et al. *Development of a power system model For Strathfarm*. English. EERA DeepWind 2023 ; Conference date: 18-01-2023 Through 20-01-2023. Jan. 2023.

[33] A. Stock et al. *Wind Farm Control at the University of Strathclyde, Past, Present and Future*. 2019.

[34] A. Peña, P.R. Réthoré, and M.P. van der Laan. "On the application of the Jensen wake model using a turbulence-dependent wake decay coefficient: the Sexbierum case". In: *Wind Energy* 19.4 (2016), pp. 763–776.

[35] S. Frandsen et al. "Analytical modelling of wind speed deficit in large offshore wind farms". In: *Wind Energy: An International Journal for Progress and Applications in Wind Power Conversion Technology* 9.1-2 (2006), pp. 39–53.

[36] S. Alaoui-Sosse, D. Pierre, and P. Médina. “In Situ Observations of Wind Turbines Wakes with Unmanned Aerial Vehicle BOREAL within the MOMEMTA Project”. In: *Atmosphere* 13.5 (2022), p. 775.

[37] M.D. Cole, W.E. Leithead, and O. Anaya-Lara. *Investigation and Assessment of the Benefits For Power Systems From Wind Farm Control*. 2023.

Appendices

Appendix A: Access Modelling

Tables

Forecasting Percentage Errors

Model	P_0	Expected Delay
Empirical	0.352	41.6
DTMC	0.352	22.7
Markov Weather	60.6	19.7
MLE	33.1	91.2
Moments	0.477	84.5

Table 2: Annual

Model	P_0	Expected Delay
Empirical	2.97	45.6
DTMC	2.97	26.6
Markov Weather	77.6	15.3
MLE	94.7	53.6
Moments	38.6	90.4

Table 3: Biannual

Model	P_0	Expected Delay
Empirical	20.6	27.0
DTMC	20.6	31.2
Markov Weather	74.5	35.2
MLE	151.1	91.0
Moments	8.1	88.8

Table 4: Seasonal

Model	P_0	Expected Delay
Empirical	29.3	47.3
DTMC	29.3	55.0
Markov Weather	6.76	1.64e7
MLE	11.3	1.16e6
Moments	3.21	7.32e6

Table 5: Month-Separated

Code

Functions

```
# Import essential packages
from scipy.stats import norm
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import weibull_min
from scipy.stats import lognorm
from scipy.special import gamma
from scipy.special import gammainc
from scipy.integrate import quad
from scipy.optimize import minimize
from statsmodels.distributions.empirical_distribution import ECDF
from scipy.stats import chisquare
from sklearn.model_selection import train_test_split
from ipynb.fs.full.Functions import *


#-------------------------------------------------------------------------------------------
#**********************************Continuous Models**************************************
#-------------------------------------------------------------------------------------------
class Continuous_Model:
    """
    Continuous models class for analyzing access probability and expected delays.

    """

    def __init__(self, data, H_th, t_req, method='Moments'):
        """
        Initializes a WaveProbs object.

        Args:
            data (DataFrame): Input data containing wave characteristics.
            H_th (float): Threshold for significant wave height.
            t_req (float): Required access window in hours.
            method (str, optional): Method for fitting distributions ('Moments' or other).
                                    Defaults to 'Moments'.
        """

        self.H_th = H_th # Threshold for significant wave height
        self.data = data # Input data for analysis
        self.t_req = t_req # Required access window
        self.method = method # Method for fitting distributions ('Moments' or other)

        self.data_setup() # Set up data for analysis
        self.fit_weibulls() # Fit Weibull distributions

        # Set up Weibull parameters and features
        [k_H, H_0, H_c] = self.weibull_H
        self.H_m = weibull_min.mean(k_H, H_0, H_c)
        self.P_H = 1 - weibull_min.cdf(self.H_th, k_H, H_0, H_c)
        self.calm_statistics() # Calculate calm-related statistics
        self.storm_statistics() # Calculate storm-related statistics

        # Calculate the outputs
        self.P_0 = self.calc_P_0(self.t_req) # Probability of instant access
```

```
    self.ed_total = self.calc_expected_delays(self.t_req) # Calculate expected delays


#-----------------------------------------------------------------------------------------------------
# Initialisation functions
#-----------------------------------------------------------------------------------------------------
def data_setup(self):
    """
    Set up data sets for storm times and calm times to fit Weibull distributions.
    """

    # Reset index and create new columns for analysis
    df = self.data
    df['Hours'] = [i for i in range(len(df))]

    # Determine status of wave conditions and detect changes
    df['Status'] = df.H >= self.H_th
    df['Change'] = df.Status.diff()
    change_df = df[df.Change == True]

    # Calculate times between changes and identify starting condition
    change_times = [0] + [time for time in change_df.Hours]
    times_between = np.diff(change_times)
    start_condition = 'n' if int(df.Status[0]) == 0 else 'x'
    alt_condition = 'x' if start_condition == 'n' else 'n'

    # Create a dictionary to store distribution data
    distribution_data = {'H': df.H}
    distribution_data[str(start_condition)] = [time for time in times_between[0::2]]
    distribution_data[str(alt_condition)] = [time for time in times_between[1::2]]

    # Update attributes
    self.data = df
    self.distribution_data = distribution_data

def fit_weibulls(self):
    """
    Fit Weibull distributions to calm times, storm times, and significant wave height.
    """

    def moment(x, k):
        x = [0] + np.sort(x)
        n = len(x)
        return np.sum([(1 - r / (n - 1)) ** k * (x[r + 1] - x[r]) for r in range(n - 1)])

    def moments_fit(x):
        # Calculate moments to estimate shape, loc, and scale parameters
        m_1 = np.mean(x)
        m_2 = moment(x, 2)
        m_3 = moment(x, 3)
        m_4 = moment(x, 4)
        shape = np.log(2) / (np.log(m_1 - m_2) - np.log(m_2 - m_4))
        loc = 0
        scale = (m_1 - loc) / gamma(1 + 1 / shape)
        return shape, loc, scale

    if self.method == 'Moments':
        # Fit Weibull distributions using moment-based estimation
        self.weibull_H = moments_fit(self.distribution_data['H'])
```

```
            self.weibull_n = moments_fit(self.distribution_data['n'])
            self.weibull_x = moments_fit(self.distribution_data['x'])
        else:
            # Fit Weibull distributions using Maximum Likelihood Estimation (MLE)
            self.weibull_H = weibull_min.fit(self.distribution_data['H'])
            self.weibull_n = weibull_min.fit(self.distribution_data['n'], floc=0)
            self.weibull_x = weibull_min.fit(self.distribution_data['x'], floc=0)

        # Store the fitted distributions for future reference
        self.distributions = {'H': self.weibull_H, 'n': self.weibull_n, 'x': self.weibull_x}

    #-----------------------------------------------------------------------------------------------
    # Calculation of calm distrubution based statistics
    #-----------------------------------------------------------------------------------------------
    def calm_statistics(self):
        """
        Calculate calm-related statistics based on fitted Weibull distribution parameters.
        """

        # Extract Weibull parameters for calm distribution
        (self.alpha_n, self.loc_n, self.beta_n) = self.weibull_n

        # Calculate parameters and statistics for the calm distribution
        self.g_n = (1 + 1 / self.alpha_n) ** self.alpha_n
        self.tau_n = weibull_min.mean(self.alpha_n, loc=self.loc_n, scale=self.beta_n)
        self.q_n = self.calc_q_n(self.t_req)
        self.M_qn = self.calc_M_qn(self.t_req)
        self.Q_n = self.calc_Q_n(self.t_req)
        self.M_qqn = self.calc_M_qqn(self.t_req)

    def calc_M_qn(self, T):
        """
        Calculate the normalized expected value of calm duration up to time T.

        Args:
            T (float): Time value.

        Returns:
            float: Normalized expected value of calm duration up to time T.
        """
        integral = quad(lambda t: t * self.calc_q_n(T), 0, T)
        I = integral[1] - integral[0]
        return 1 / self.tau_n * I

    def calc_Q_n(self, T):
        """
        Calculate the probability of no calm events up to time T.

        Args:
            T (float): Time value.

        Returns:
            float: Probability of no calm events up to time T.
        """
        return np.exp(-self.g_n * (T / self.tau_n) ** self.alpha_n)

    def calc_q_n(self, T):
        """
        Calculate the intensity function for calm events at time T.
```

```
    Args:
        T (float): Time value.

    Returns:
        float: Intensity function for calm events at time T.
    """
    return self.g_n * self.alpha_n / self.tau_n * (T / self.tau_n) ** (self.alpha_n - 1) *
        self.calc_Q_n(T)

def calc_M_qqn(self, T):
    """
    Calculate the normalized second moment (calm) assuming Weibull distribution.

    Args:
        T (float): Time value.

    Returns:
        float: Normalized second moment (calm) up to time T.
    """
    return gammainc(1 + 2 / self.alpha_n, self.g_n * (T / self.tau_n) ** self.alpha_n) / (2 *
        gamma(1 + 1 / self.alpha_n) ** 2)

#-----------------------------------------------------------------------------------------------------
# Calculation of storm distribution based statistics
#-----------------------------------------------------------------------------------------------------

def storm_statistics(self):
    """
    Calculate storm-related statistics based on fitted Weibull distribution parameters.
    """

    # Extract Weibull parameters for storm distribution
    (self.alpha_x, self.loc_x, self.beta_x) = self.weibull_x

    # Calculate parameters and statistics for the storm distribution
    self.g_x = (1 + 1 / self.alpha_x) ** self.alpha_x
    self.tau_x = weibull_min.mean(self.alpha_x, loc=self.loc_x, scale=self.beta_x)
    self.q_x = self.calc_q_x(self.t_req)
    self.M_qx = self.calc_M_qx(self.t_req)
    self.Q_x = self.calc_Q_x(self.t_req)
    self.M_qqx = self.calc_M_qqx(self.t_req)

def calc_M_qx(self, T):
    """
    Calculate the normalized expected value of storm duration up to time T.

    Args:
        T (float): Time value.

    Returns:
        float: Normalized expected value of storm duration up to time T.
    """
    integral = quad(lambda t: t * self.calc_q_x(T), 0, T)
    I = integral[1] - integral[0]
    return 1 / self.tau_x * I

def calc_Q_x(self, T):
    """
```

```
    Calculate the probability of no storm events up to time T.

    Args:
        T (float): Time value.

    Returns:
        float: Probability of no storm events up to time T.
    """
    return np.exp(-self.g_x * (T / self.tau_x) ** self.alpha_x)

def calc_q_x(self, T):
    """
    Calculate the intensity function for storm events at time T.

    Args:
        T (float): Time value.

    Returns:
        float: Intensity function for storm events at time T.
    """
    return self.g_x * self.alpha_x / self.tau_x * (T / self.tau_x) ** (self.alpha_x - 1) *
        self.calc_Q_x(T)

def calc_M_qqx(self, T):
    """
    Calculate the normalized second moment (storm) assuming Weibull distribution.

    Args:
        T (float): time needed to carry out operation

    Returns:
        float: Normalized second moment (storm) up to time T.
    """
    return gammainc(1 + 2 / self.alpha_x, self.g_x * (T / self.tau_x) ** self.alpha_x) / (2 *
        gamma(1 + 1 / self.alpha_x) ** 2)

#-----------------------------------------------------------------------------------------------------
# Delay calculations
#-----------------------------------------------------------------------------------------------------

def calc_expected_delays(self,T=None):
    '''
    Calculate expected delays

    Args:
        T (float): time needed to carry out operation

    Returns:
        float: Number of hours until expected window of length t
    '''
    P_H = self.P_H
    tau_x = self.tau_x
    tau_n = self.tau_n
    if T == None:
        T = self.t_req
        M_qqx = self.M_qqx
        M_qqn = self.M_qqn
        Q_n = self.Q_n
        M_qn = self.M_qn
```

```
        else:
            M_qqx = self.calc_M_qqx(T)
            M_qqn = self.calc_M_qqn(T)
            Q_n = self.calc_Q_n(T)
            M_qn = self.calc_M_qn(T)

        self.ed_1 = P_H*M_qqx*tau_x # Expected delay time of type 1
        self.ed_2a = (1-P_H)**2/P_H*M_qqn*tau_x # Expected delay time of type 2a
        self.ed_2b = P_H*Q_n*T**2/(2*tau_x) # Expected delay time of type 2b
        self.ed_3 = (1-P_H*M_qn)*tau_x*P_H*Q_n*T # Expected delay time of type 3
        self.ed_higher = (P_H+(1-P_H)*M_qn+P_H*Q_n*T/tau_x)*(P_H+(1-P_H)*M_qn)*tau_x/(P_H*Q_n)
        self.ed_total = self.ed_1+self.ed_2a+self.ed_2b+self.ed_3+self.ed_higher # Calculate total
            expected delay as sum
        return self.ed_total

    def calc_P_0(self,T=None):
        '''
        Calculate expected probability of instant access

        Args:
            T (float): time needed to carry out operation
        Returns:
            float: probability of instant access

        '''
        if T == None:
            return (1-self.P_H)*(1-self.M_qn-self.Q_n*self.t_req/self.tau_n)
        else:
            return (1-self.P_H)*(1-self.calc_M_qn(T)-self.calc_Q_n(T)*T/self.tau_n)
    #-----------------------------------------------------------------------------------------------------
    # Graphs and other Outputs
    #-----------------------------------------------------------------------------------------------------

    def compare_fit(self, var='n'):
        """
        Compare the fitted Weibull distribution with empirical data for a specified variable.

        Args:
            var (str): Variable to compare ('n' for calm duration, 'x' for storm duration, 'H' for
                significant wave height).

        Returns:
            None
        """

        # Extract data and distribution parameters
        data = self.distribution_data[var]
        shape, loc, scale = self.distributions[var]

        # Generate x values for the fitted Weibull distribution
        x = np.linspace(weibull_min.ppf(0.001, shape, loc=loc, scale=scale),
                        weibull_min.ppf(0.999, shape, loc=loc, scale=scale), 100)

        # Plot the fitted Weibull distribution and empirical data
        plt.plot(x, weibull_min.cdf(x, shape, loc=loc, scale=scale), 'b--', label='Fitted Weibull
            Distribution')
        x = np.linspace(min(data), max(data), 100)
        plt.ylabel('Probability Density')
```

```
        # Set x-axis label based on the variable
        if var == 'H':
            xlab = r'Significant Wave Height, $H_s$ (m)'
        elif var == 'n':
            xlab = 'Calm Duration (h)'
        else:
            xlab = 'Storm Duration (h)'

        plt.xlabel(xlab)
        ecdf = ECDF(data)
        plt.scatter(x, ecdf(x), c='k', s=2, label='Empirical Data')
        plt.xlim(min(x), max(x))
        plt.ylim(0, 1)
        plt.grid()
        plt.legend()

    def compare_fits(self):
        """
        Compare fitted Weibull distributions for multiple variables using subplots.

        Args:
            None

        Returns:
            None
        """

        # Create subplots for each variable
        fig, ax = plt.subplots(1, 3, figsize=(15, 2.5))

        for i, var in enumerate(['H', 'n', 'x']):
            sca = ax[i]
            data = self.distribution_data[var]
            shape, loc, scale = self.distributions[var]

            # Generate x values for the fitted Weibull distribution
            x = np.linspace(weibull_min.ppf(0.001, shape, loc=loc, scale=scale),
                            weibull_min.ppf(0.999, shape, loc=loc, scale=scale), 100)

            # Plot the fitted Weibull distribution and empirical data
            sca.plot(x, weibull_min.cdf(x, shape, loc=loc, scale=scale), 'b--', label='Fitted
                Weibull Distribution')
            x = np.linspace(min(data), max(data), 100)

            # Set x-axis label based on the variable
            if var == 'H':
                xlab = r'Significant Wave Height, $H_s$ (m)'
            elif var == 'n':
                xlab = 'Calm Duration (h)'
            else:
                xlab = 'Storm Duration (h)'

            sca.set(xlabel=xlab)

            # Set y-axis label for the first subplot only
            if i == 0:
                sca.set(ylabel='Probability Density')

            ecdf = ECDF(data)
```

```
            sca.scatter(x, ecdf(x), c='k', s=2, label='Empirical Data')
            sca.set_xlim(min(x), max(x))
            sca.set_ylim(0, 1)
            sca.grid()
            sca.legend()

    def time_effect(self, Ts):
        """
        Visualize the effect of different access window lengths on probability and expected delays.

        Args:
            minimum (float): Minimum access window length.
            maximum (float): Maximum access window length.

        Returns:
            None
        """

        fig, ax = plt.subplots(1, 2, figsize=(15, 2.5))
        sca = ax[0]
        probs = [self.calc_P_0(t) for t in Ts]
        sca.plot(Ts, probs, 'k--')
        sca.set_xlim(np.min(Ts), np.max(Ts))
        sca.set(xlabel=r'Required Access Window, $t$ (h)', ylabel='Probability of Instant Access')
        plt.grid()

        sca = ax[1]
        eds = [self.calc_expected_delays(t) for t in Ts]
        sca.plot(Ts, eds, 'k--')
        sca.set_xlim(np.min(Ts), np.max(Ts))
        sca.set(xlabel=r'Required Access Window, $t$ (h)', ylabel='Expected Delay (h)')
        plt.grid()

    def height_effect(self, minimum, maximum):
        """
        Visualize the effect of different access window lengths on probability based on height.

        Args:
            minimum (float): Minimum access window length.
            maximum (float): Maximum access window length.

        Returns:
            None
        """

        ts = np.arange(minimum, maximum, (maximum - minimum) / 100)
        probs = [self.calc_P_0(t) for t in ts]
        plt.plot(ts, probs, 'k--')
        plt.xlim(minimum, maximum)
        plt.xlabel(r'Required Access Window, $t$ (h)')
        plt.ylabel('Probability of Instant Access')
        plt.grid()

    # ... (other methods)


#-------------------------------------------------------------------------------------
#*********************************Discrete Models*****************************************
```

```
#-------------------------------------------------------------------------------------------
class Discrete_Model:
    '''
    Class using a Discrete-Time Markov Chain to calculate offshore accessibility statistics
    '''
    def __init__(self,data, H_th, t_req, method='Access'):
        '''
        @param data: csv, dataframe with datetime column labelled 'Time' and waveheight labelled 'H'
        @param H_th: float, wave height in metres
        @param t_req: int, required time to carry out operation in hours
        @param method: str, choice of 'Access' or 'Weather' to determine which states are being
            modelled
        '''
        # Initialise input variables to be class attributes
        self.H_th = H_th
        self.data = data
        self.t_req = t_req
        self.method = method

        # Run set-up functions
        self.data_setup()
        self.calculate_probs()

        # Calculate the outputs using defined class functions
        self.P_0 = np.mean(self.states)
        self.ed_total = self.calc_expected_delay(self.t_req)


    #-----------------------------------------------------------------------------------------------
    # Initialisation functions
    #-----------------------------------------------------------------------------------------------
    def data_setup(self):
        '''
        Sets up data sets for storm times and calm times to fit Weibulls
        '''

        df = self.data

        # Set up boolean column of whether weather conditions are stormy (if threshold wave height
            exceeded)
        df['Storm'] = df.H>=self.H_th

        # Calculate probabilities of each weather state as the proportion of time spent in each
            state
        self.P_s = np.mean(df.Storm)
        self.P_c = 1-self.P_s

        # Define the time required to carry out op.
        T = self.t_req

        # Initialise a vector of ones for instant access
        instant_access = np.ones(len(df)-T)
        # For each time i...
        for i in range(len(df)-T):
            # For each of the next T time steps after time i...
            for j in range(T):
                #...if time i+j is stormy there's no access at time i so set access status=0 and
                    move to next time
                if df.Storm[i+j]:
```

```
                instant_access[i] = 0
                break
            # otherwise we complete T consecutive calm period and keep state as 1

    # Append 0s to the end of instant_access list to equalize length with data
    df['instant_access'] = list(instant_access) + [0]*T

    # Define states attribute to be weather or accessibility state dependent on chosen method
    self.states = df.Storm if self.method=='Weather' else [int(d) for d in instant_access]
    self.instant_access = [int(d) for d in instant_access]

    # Define n to be number of recorded time steps
    n = len(df)
    # Define zero vector to store delay times
    delay = np.zeros(n)
    # For each time i backwards from second to last time...
    for i in range(n-2,-1,-1):
        # If access hasn't yet occurred increment delay time by 1 from next chronological element
        if df.instant_access[i] == 0:
            delay[i] = delay[i+1]+1
        # Otherwise reset delay to 0 as we have access at that point
        else:
            delay[i] = 0

    # Insert delays as a column in the dataframe
    df['delay'] = delay

    # Redefine data attribute with updated dataframe
    self.data = df

    # Locate last access point as points after this are considered censored in terms of delay
    last_access = df[df['instant_access']==1].index[-1]
    # Set class attribute as list of delay times.
    self.delays = list(df.delay[:last_access])

def calculate_probs(self):
    '''
    Calculate transition matrix and stationary probabilities for DTMC
    '''
    # Calculate transition matrix as observed probs
    self.m = transition_matrix(self.states)
    # Calculate stationary probabilities as eigenvector of the transition matrix
    self.pi = np.linalg.eig(self.m)[1][0]

def calc_P_0(self,T=None, return_states = False):
    '''
    Calculate instant_access probability
    '''

    # If no time selected set T to be the initial input required op time
    if T==None:
        T = self.t_req

    # Convert T from float to integer to use as index
    T = int(T)

    # Copy data
    df = self.data
```

```
        # If using Weather chain...
        if self.method == 'Weather':
            #...calculate instant access probability as chance of calm state*chance of staying for
                T-1 consecutive
            p = self.P_c*self.m[0][0]**(T-1)

        # Otherwise using Access chain...
        else:
            # Calculate as mean of observed instant access (equivalent to calc through Markov chain)
            if T == self.t_req:
                instant_access = self.instant_access
            else:
                instant_access = np.ones(len(self.data)-T)
                # For each time i...
                for i in range(len(self.data)-T):
                    # For each of the next T time steps after time i...
                    for j in range(T):
                        #...if time i+j is stormy there's no access at time i so set access status=0
                            and move to next time
                        if self.data.Storm[i+j]:
                            instant_access[i] = 0
                            break
                        # otherwise we complete T consecutive calm period and keep state as 1

            # Calculate p as the mean of instant access (proportion spent in state 1)
            p = np.mean(instant_access)

            # Return tuple including states if return_states=True
            if return_states:
                ret = (p, instant_access)
            else:
                ret = p

        return ret

    def calc_expected_delay(self,T=None):
        '''
        Calculated expected delay time
        '''
        # If no T inputted use original input op time
        if T == None:
            T = self.t_req

        # If using weather method...
        if self.method == 'Weather':
            #... solve simultaneous equations of recursive expected times (see notes)

            # Set up transition probs
            P_cc = self.m[0][0]
            P_cs = self.m[0][1]
            P_sc = self.m[1][0]
            P_ss = self.m[1][1]

            # Set vectors b and x as defined in simultaneous equations
            b = [1]*T
            x = np.zeros(T)

            # Set TxT identity matrix
            a = np.eye(T)
```

```
            # Convert to the matrix A in defined simultaneous equations
            a[:,0] = np.array([1-P_ss]+[-P_cs]*(T-1))
            s = np.arange(len(a))
            a[s[:-1], s[1:]] = [-P_cc]*(T-1)
            a[0,1] = -P_sc

            # Solve equations
            x = np.linalg.solve(a,b)
            # Return expected delay as sum of product of exp wait from each state weighted by their
                resp starting prob
            return self.P_c*x[0]+self.P_s*x[1]

        else:
            # Otherwise if using Access method multiply expected wait time from state 0 (1/P0,1) by
                its starting prob
            # If T is originally inputted operation. time use existing attributes
            if T == self.t_req:
                return (1-np.mean(self.states))/self.m[0][1]
            else:
            # Otherwise recalculate states and probability to calculate
                (p, states) = self.calc_P_0(T, return_states=True)
                states = [int(d) for d in states]
                m = transition_matrix(states)
                return (1-np.mean(states))/m[0][1]

    def time_effect(self, Ts, plot='Scatter'):
        '''
        Compare access probability and delay times for varying operation time
        '''
        # Set up 2 empty subplots
        fig, ax = plt.subplots(1,2, figsize=(15,2.5))
        # On first set of axes...
        sca = ax[0]
        # Calculate access prob for each operation time
        prob = [self.calc_P_0(t) for t in Ts]
        # Plot operation time against prob
        # If plot option is scatter produce scatter plot
        if plot =='Scatter':
            sca.scatter(Ts,prob,s=5)
        # Otherwise line plot
        else:
            sca.plot(Ts,prob,'k--')
        # Set x limit to maximum operation time
        sca.set_xlim(min(Ts), max(Ts))
        # Label x and y axes
        sca.set(xlabel=r'Required Access Window, $t$ (h)',ylabel='Probability of Instant Access')
        # Add grid
        plt.grid()

        # On second set of axes...
        sca = ax[1]
        # Calculate expected delay for each operation time
        ed = [self.calc_expected_delay(t) for t in Ts]
        # Plot operation time against expected delay
        # If option is scatter produce scatter plot
        if plot =='Scatter':
            sca.scatter(Ts,ed,s=5)
        # Otherwise line plot
        else:
```

```
        sca.plot(Ts,ed,'k--')
    # Set x axis limit to be the maximum operation time
    sca.set_xlim(min(Ts),max(Ts))
    # Label x and y axes
    sca.set(xlabel=r'Required Access Window, $t$ (h)',ylabel='Expected Delay (h)')
    plt.grid()

def CIs(self, val='Delay', opt='Markov', remove_extremes=False, alpha=0.05, split=1,
    delay_method='Obs'):
    '''
    Calculate extreme values
    '''
    # Transition matrix
    m = self.m
    # Data set
    data = self.data
    # States of access
    states = data.instant_access
    # Standard normal alpha%/1-alpha% quantile
    z = norm.ppf(1-alpha)
    # Size of dataset
    n = len(data)

    def markov_m_CI(states, m, alpha=0.05, i=0, j=1):
        '''
        Calculate CI for transition matrix probabilities
        '''
        # Define original transition prob
        p_ij = m[i][j]
        # Calculate total number of state i
        f = np.sum(states) if i==1 else len(states)-np.sum(states)
        # Calculate normal quantile
        z = norm.ppf(1-alpha/2)
        # Calculate lower and upper CI limit as defined by Crow (1979)
        lower = p_ij-z*np.sqrt((p_ij*(1-p_ij))/f)
        upper = p_ij+z*np.sqrt((p_ij*(1-p_ij))/f)
        # Flip CI if lower limit is higher than upper limit
        if lower > upper:
            (lower, upper) = (upper, lower)
        return lower,upper

    def markov_p_CI(states, m, alpha = 0.05):
        '''
        Calculate CI for base probabilities of state 1 (probability of instant access)
        '''
        # Define normal quantile
        z = norm.ppf(alpha/2)
        # Calculate number of accessible states
        S = np.sum(states)
        # Calculate point estimate for prob of instant access as mean proportion of time spent
            in state 1
        p_hat = np.mean(states)
        # Calculate the product-moment correlation coefficient between adjacent states
        theta_hat = (m[1][1]-p_hat)/(1-p_hat)

        # For small values of P_0 or P_1 calculate the Anderson-Burstein CI as...
        if p_hat<0.1 or p_hat>0.9:
            # Define the adjustment factor
            d = 0.5
```

```
            # Calculate Pearson-Hartley Poisson CI lower and upper limits (Pearson and Hartley
                1966)
            L = S-z*np.sqrt(S/n)
            U = S+z*np.sqrt(S/n)
            # Calculate the Modified Anderson-Burstein CI using base Pearson-Hartley Poisson CI
                (Crow 1979)
            p_ui = U/(n+d+0.5*(U-S))
            p_li = L/(n-0.5*(S-1-L))
            # Calculate adjusted MAB interval for improved accuracy (Bedrick and Aragon 1989)
            lower =
                p_hat-(p_hat-p_li)*(1+2*theta_hat/(1-theta_hat))**0.5*(1-p_hat)**0.5/(1-0.5*p_hat)
            upper =
                p_hat+(p_ui-p_hat)*(1+2*theta_hat/(1-theta_hat))**0.5*(1-p_hat)**0.5/(1-0.5*p_hat)

        # For all other values of P_0 & P_1 calculate the CI using the Crow pivotal sample
            proportion method...
        else:
            # Calculate statistic h as a function of product moment correlation coefficient
            h = 1+2*theta_hat/(1-theta_hat)*(1-(1-theta_hat**n)/(n*(1-theta_hat)))
            # Calculate lower and upper CI using sample proportion method (Crow 1979)
            lower = (S+0.5*z**2+(0.5+z*np.sqrt(h*(S+0.5-(S+0.5)**2/n+0.25*z**2*h))))/(n+z**2*h)
            upper = (S-0.5*z**2+(0.5+z*np.sqrt(h*(S-0.5-(S-0.5)**2/n+0.25*z**2*h))))/(n+z**2*h)

        # Flip CI if lower limit is greater than upper limit
        if lower > upper:
            (lower, upper) = (upper, lower)

        return (lower,upper)

    def markov_delay_CI(data, m, alpha=0.05, remove_extremes=False, delay_method='Obs'):
        '''
        Calculate CI for Expected Delay
        '''
        # Calculate 1-alpha% standard normal quantile
        z = norm.ppf(1-alpha)
        # Define states of accessibility
        states = data.instant_access
        # Calculate the confidence interval of instant access probabability
        p_ci = markov_p_CI(states, m, alpha)
        m_ci = markov_m_CI(states, m)
        n = len(data)

        # Calculate expectation and variance of p using confidence interval
        exp_p = 1-np.mean(p_ci)
        se_p = np.diff(p_ci)[0]/(2*z)
        var_p = np.var(states)
        print(f'Var p: {var_p}')
        print(f'Exp p: {exp_p}')
        # If using Taylor approach, calculate lambda in terms of the transition probability m...
        if delay_method=='Taylor':
            # Calculate expectation and variance of m using confidence interval
            exp_m = (m_ci[1]+m_ci[0])/2
            var_m = ((m_ci[1]-m_ci[0])/(2*z))**2*len(data)**2
            # Use Taylor series approximations to calculate the expectation and variance of
                lambda (1/m)
            exp_lambda = 1/exp_m
            var_lambda = 1/((1-exp_m)**4)*var_m
            print(f'Var m: {var_m}')
            print(f'Exp m: {exp_m}')
```

```
            print(f'Var lam: {var_lambda}')
            print(f'Exp m: {exp_lambda}')
        # If using observed data instead of approximation...
        elif delay_method=='Obs':
            # Define data as list of expected wait times from inaccessible states
            d = data[data['Storm']==0].delay[:data[data['instant_access']==1].index[-1]]
            # If dataset heavily affected by outliers can opt to remove (remove data points
                further than 2 s.ds away)
            if remove_extremes:
                d = remove_outliers(d)
            # Calculate expectation and variance of lambda from observed data
            exp_lambda = np.mean(d)
            var_lambda = np.var(d)
        else:
            print('INVALID METHOD FOR DELAY CI CALCULATION')


        # Combine variances using Taylor series method (Wolter 2006)
        var_c = var_p*var_lambda+var_p*exp_lambda**2*var_lambda*exp_p**2
        # Define centre of confidence interval as the expected calculated value
        centre = self.ed_total
        # If applying CI to a smaller future set variance is in terms of a smaller data length
        if split!=1:
            n = n*(1-split)/split
        # Calculate standard error using variance estimator
        se = np.sqrt(var_c/n)
        # Define upper and lower CI limits
        lower = pos(centre-z*se) # Define as positive (i.e. above 0) since negative delay not
            possible
        upper = pos(centre+z*se)

        # Flip CI if lower limit is greater than upper limit
        if lower>upper:
            (lower, upper) = (upper, lower)
        return (lower,upper)

    # If using Markov method...
    if opt == 'Markov':
        # If calculating CI for delay time...
        if val=='Delay':
            # Calulate delay CI
            return markov_delay_CI(data, m, alpha, remove_extremes, delay_method)
        # Otherwise calculating CI for prob of instant access...
        else:
            # Calculate p_0 CI
            return markov_p_CI(states, m, alpha)
    # Otherwise calculate empirically...
    else:
        # If calculating delay CI...
        if val=='Delay':
            # Calc delay CI using sample mean and variance
            u = np.mean(self.delays)
            v = np.var(self.delays)
            return (u-z*np.sqrt(v/n), u+z*np.sqrt(v/n))
        # Otherwise calculating instant access prob CI...
        else:
            # Calc instant access prob CI using sample mean and variance
            u = np.mean(states)
            v = np.var(states)
```

```
            return (u-z*np.sqrt(v/n), u+z*np.sqrt(v/n))

    def empirical(self,T=None, H=None, remove_extremes=False):
        '''
        Calculate the empirical access probability and expected delay
        '''
        # Set T_req and H_th as original inputs if not specified for function
        if T == None:
            T = self.t_req
        if H == None:
            H = self.H_th

        # Calculate P_0 as the mean of instant access status (proportion spent in state 1)
        P_0 = np.mean(self.instant_access)


        # If extreme values problematic in data set...
        if remove_extremes:
            # ...remove delay times more than 2 standar deviations from mean delay time
            delays = remove_outliers(self.delays)
        else:
            # otherwise use all delay times
            delays = self.delays

        # Calculate mean delay time
        ed_total = np.mean(delays)

        return (P_0, ed_total)



#-------------------------------------------------------------------------------------------
#**********************************Other Functions*****************************************
#-------------------------------------------------------------------------------------------
def remove_outliers(d):
    '''
    Removes elements in data more than 2 standard deviations away from sample mean
    '''
    u = np.mean(d)
    s = np.std(d)
    return [el for el in d if np.abs(el-u)<2*s]

def pos(num):
    '''
    Returns positive part of a real number
    '''
    return num if num>0 else 0

def transition_matrix(states, num_states = 2):
    '''
    Calculate observed transition matrix
    '''
    # Define number of states
    n = num_states

    # Make square matrix
    M = [[0]*n for i in range(n)]

    # For pairs of elements and future elements...
```

```
    for (i,j) in zip(states,states[1:]):
        # ...increment transition matrix to create frequency matrix of transitions
        M[i][j] += 1

    # For each row...
    for row in M:
        # Sum the row...
        s = sum(row)
        # Assuming row is nonzero...
        if s > 0:
            # Divide frequencies by row sums to get probability matrix
            row[:] = [f/s for f in row]
    return M
```

Appendix B: Wake Modelling

Variable Glossary

$a =$ Induction factor
$a_F = 2a$
$c =$ Scaling Coefficient
$C_{f1} =$ Characteristic flow speed parameter
$C_{f2} =$ Characteristic flow speed parameter
$C_T =$ Thrust Coefficient
$d_0 =$ Initial wake diameter $[m]$
$d =$ Variable wake diameter $[m]$
$D =$ Rotor diameter $[m]$
$I =$ Turbulence intensity
$K =$ Wake spread parameter
$l =$ Longitudinal Offset
$\bar{l} =$ Normalised Longitudinal Offset
$r_0 =$ Initial wake radius $[m]$
$r =$ Variable wake radius $[m]$
$\bar{r} =$ Normalised Radial Distance
$R =$ Rotor radius $[m]$
$\rho =$ Air density $kgm^{-3}]$
$\sigma =$ Gaussian wake standard deviation
$T =$ Torque $[N - m]$
$\theta =$ Azimuthal Angle $[^\circ]$
$r =$ Radial Distance $[m]$
$U_W =$ Minimum wake speed $[ms^{-1}$
$v_0 =$ Initial wake wind speed (immediately behind rotor) $[ms^{-1}]$
$v = U =$ Variable wind speed $[ms^{-1}]$
$V_m =$ Minimum Wake Velocity $[ms^{-1}]$
$\delta v =$ Wake Deficit
$x =$ Lateral displacement $[m]$
$y =$ Longitudinal displacement $[m]$
$z =$ Vertical displacement $[m]$

Reduction of Coefficients

Let us consider the integral

$$I = \int_0^{2\pi} \exp\left(\frac{lr\sin(\theta)}{\sigma^2}\right)\cos(n\theta)d\theta.$$

First let us write $k = \frac{lr}{\sigma^2}$. Then I may be rewritten as

$$I = \int_0^{2\pi} \exp\big(k\sin(\theta)\big)\cos(n\theta)d\theta$$

cos

By multiple angle formulae rules we have

$$\cos(n\theta) = \begin{cases} (-1)^{\frac{(n-1)}{2}} \cos(\theta) U_{n-1}\left(\sin(x)\right) & \text{if } n \text{ is even} \\ (-1)^{\frac{(n-1)}{2}} T_{n-1}\left(\sin(x)\right) & \text{if } n \text{ is even} \end{cases},$$

where T_n and U_{n-1} are Chebyshev polynomials of the first and second kind respectively. It follows that for odd n

$$\int_0^{2\pi} e^{k\sin(\theta)} \cos(n\theta) d\theta = \int_0^{2\pi} e^{k\sin(\theta)} (-1)^{\frac{n-1}{2}} \cos(\theta) U_{n-1}\left(\sin(\theta)\right) d\theta.$$

Let $u = \sin(\theta)$, then $du = \cos(\theta)d\theta$ and $d\theta = \frac{du}{\cos(\theta)}$. We may then rewrite the integral as

$$\int_0^0 e^{ku} (-1)^{\frac{n-1}{2}} U_{n-1}(u) du.$$

Thus $I = 0$, and therefore $a_n = 0$, for all odd n.

Additionally, the majority of a_n, was equal or close to zero. As proposed by the previous fit, a_0 and a_2 were the only taken values.

Exponential Integral Model

For remaining values, the relationship between $k = \frac{lr}{\sigma^2} = kr$ and I may be modelled using a exponential fit via least square regression. The choice of this fit was determined through exploratory data analysis. The values for I in terms of k are as follows

$$I|_{n=0} = -0.445e^{0.82kr} - 3.21 \tag{80}$$

$$I|_{n=2} = 0.0846e^{-0.89kr} - 1.45 \tag{81}$$

Thus the coefficients may be calculated as

$$a_n = \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2 \sigma R^{c+1}} \int_0^R \left(ae^{\frac{blr}{\sigma^2}} + c\right) e^{\frac{-r^2}{2\sigma^2}} r^c dr \tag{82}$$

$$= \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2 \sigma R^{c+1}} \left[a \int_0^R e^{\frac{2blr - r^2}{2\sigma^2}} r^c dr + c \int_0^R e^{\frac{-r^2}{2\sigma^2}} r^c\right] \tag{83}$$

$$= \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2 \sigma R^{c+1}} \left[aI_1 + cI_2\right], \tag{84}$$

where I_2 may be evaluated as

$$I_2 = \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}} \sigma^{-(c+1)}}$$

and I_1 may be integrated by parts to get

$$I_1 = e^{\frac{blr}{\sigma^2}} \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} - \frac{bl}{2(-1)^{\frac{c+1}{2}}\sigma^{1-c}} \int_0^R \Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right) e^{\frac{blr}{\sigma^2}} dr \tag{85}$$

$$\approx e^{\frac{blr}{\sigma^2}} \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} - \frac{bl}{2(-1)^{\frac{c+1}{2}}\sigma^{1-c}} \int_0^R \Gamma\left(\frac{c+1}{2}\right) \left[1 - \frac{2r^{c+1}}{\Gamma\left(\frac{c+1}{2}\right)\sigma^{c+1}(c+1)}\right] e^{\frac{blr}{\sigma^2}} dr \tag{86}$$

$$= e^{\frac{blr}{\sigma^2}} \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} - \frac{bl\Gamma\left(\frac{c+1}{2}\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{1-c}} \left[\int_0^R e^{\frac{blr}{\sigma^2}} dr - \frac{2}{\Gamma\left(\frac{c+1}{2}\right)\sigma^{c+1}(c+1)} \int_0^R e^{\frac{blr}{\sigma^2}} r^{c+1} dr\right] \tag{87}$$

$$= e^{\frac{blr}{\sigma^2}} \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} - \frac{bl\Gamma\left(\frac{c+1}{2}\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{1-c}} \left[\frac{\sigma^2}{bl} e^{\frac{blr}{\sigma^2}} - \frac{2}{\Gamma\left(\frac{c+1}{2}\right)\sigma^{c+1}(c+1)} \cdot \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\Gamma\left(c+2, -\frac{bl}{\sigma^2}r\right)}{(-1)^{c+1}}\right] \tag{88}$$

$$= e^{\frac{blr}{\sigma^2}} \frac{\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} - \frac{bl\Gamma\left(\frac{c+1}{2}\right)}{2(-1)^{\frac{c+1}{2}}\sigma^{1-c}} \left[\frac{\sigma^2}{bl} e^{\frac{blr}{\sigma^2}} - \frac{2}{\Gamma\left(\frac{c+1}{2}\right)\sigma^{c+1}(c+1)} \cdot \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\Gamma\left(c+2, -\frac{bl}{\sigma^2}r\right)}{(-1)^{c+1}}\right] \tag{89}$$

$$= \frac{1}{2(-1)^{\frac{c+1}{2}}\sigma^{-(c+1)}} \left(e^{\frac{blr}{\sigma^2}}\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right) - \sigma^4\Gamma\left(\frac{c+1}{2}\right)e^{\frac{blr}{\sigma^2}} + \frac{2\left(\frac{bl}{\sigma^2}\right)^{-c-2}\Gamma\left(c+2, \frac{-blr}{\sigma^2}\right)}{(-1)^{c+1}\sigma^{c-1}(c+1)}\right) \tag{90}$$

$$= \frac{1}{2(-1)^{\frac{c+1}{2}}\sigma^{-c-1}} \left(e^{\frac{blr}{\sigma^2}}\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right) - \sigma^4\Gamma\left(\frac{c+1}{2}\right)e^{\frac{blr}{\sigma^2}}\right) + \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-A2}\Gamma\left(c+2, \frac{-blr}{\sigma^2}\right)}{(-1)^{\frac{c+1}{2}}(c+1)} \tag{91}$$

Hence,

$$k_1 I_1 + k_2 I_2 = \left\{\frac{1}{2(-1)^{\frac{c+1}{2}}\sigma^{-c-1}} \left(\left(k_1 \cdot e^{\frac{blr}{\sigma^2}} + k_2\right)\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right) - \sigma^4\Gamma\left(\frac{c+1}{2}\right)e^{\frac{blr}{\sigma^2}}\right) + \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\Gamma\left(c+2, \frac{-blr}{\sigma^2}\right)}{(-1)^{\frac{c+1}{2}}(c+1)}\right\}_0^R \tag{92}$$

$$= \left\{\frac{k_1}{2(-1)^{\frac{c+1}{2}}\sigma^{-c-1}} \left(\left(e^{\frac{blr}{\sigma^2}} + \frac{k_2}{k_1}\right)\Gamma\left(\frac{c+1}{2}, \left(\frac{r}{\sigma}\right)^2\right) - \sigma^4\Gamma\left(\frac{c+1}{2}\right)e^{\frac{blr}{\sigma^2}}\right) + \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\Gamma\left(c+2, \frac{-blr}{\sigma^2}\right)}{(-1)^{\frac{c+1}{2}}(c+1)}\right\}_0^R \tag{93}$$

$$= \frac{k_1}{2(-1)^{\frac{c+1}{2}}\sigma^{-c-1}} \left(e^{\frac{blR}{\sigma^2}}\Gamma\left(\frac{c+1}{2}, \left(\frac{R}{\sigma}\right)^2\right) - \frac{k_2}{k_1}\gamma\left(\frac{c+1}{2}, \left(\frac{R}{\sigma}\right)^2\right) - \Gamma\left(\frac{c+1}{2}\right)\left(\sigma^4 e^{\frac{blR}{\sigma^2}} - 1 - \sigma^4\right)\right) \tag{94}$$

$$- \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\gamma\left(c+2, \frac{blR}{\sigma^2}\right)}{(-1)^{\frac{c+1}{2}}(c+1)} \tag{95}$$

and

$$a_n = \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2\sigma R^{c+1}} \int_0^R \left(ae^{\frac{blr}{\sigma^2}} + c\right) e^{\frac{-r^2}{2\sigma^2}} r^c dr \tag{96}$$

$$= \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2\sigma R^{c+1}} \left[a\int_0^R e^{\frac{2blr-r^2}{2\sigma^2}} r^c dr + c\int_0^R e^{\frac{-r^2}{2\sigma^2}} r^c\right] \tag{97}$$

$$= \frac{(c+1)V_m e^{-\frac{l^2}{2\sigma^2}}}{2\pi^2\sigma R^{c+1}} \left[\frac{k_1}{2(-1)^{\frac{c+1}{2}}\sigma^{-c-1}} \left(e^{\frac{blR}{\sigma^2}}\Gamma\left(\frac{c+1}{2}, \left(\frac{R}{\sigma}\right)^2\right) - \frac{k_2}{k_1}\gamma\left(\frac{c+1}{2}, \left(\frac{R}{\sigma}\right)^2\right) - \Gamma\left(\frac{c+1}{2}\right)\left(\sigma^4 e^{\frac{blR}{\sigma^2}} - 1 - \sigma^4\right)\right)\right. \tag{98}$$

$$\left. - \frac{\left(\frac{bl}{\sigma^2}\right)^{-c-2}\gamma\left(c+2, \frac{blR}{\sigma^2}\right)}{(-1)^{\frac{c+1}{2}}(c+1)}\right], \tag{99}$$

Similarly,

$$b_n = \frac{(c+1)V_m}{2\pi^2\sigma}\exp\left(-\frac{R^2+l^2}{2\sigma^2}\right)\left[C_0\cdot\frac{1}{c+1} + C_1\cdot\frac{lR}{\sigma^2(c+2)} + C_2\cdot\frac{l^2R^2}{\sigma^4(c+3)}\right], \tag{100}$$

where the polynomials defining the coefficients for the non-zero elements are as follows

$$I|_{n=1} = 0.123 + 1.01k + 0.780k^2 \tag{101}$$

$$I|_{n=3} = 0.0806 - 0.452k + 0.508k^2 \tag{102}$$

$$I|_{n=5} = 0.00197 - 0.00995k + 0.00811k^2 \tag{103}$$

$$I|_{n=6} = 0.000316 - 0.00154k + 0.00120k^2 \tag{104}$$

$$I|_{n=8} = 1.87e-5 - 9.29e-5k + 6.92e-5k^2. \tag{105}$$